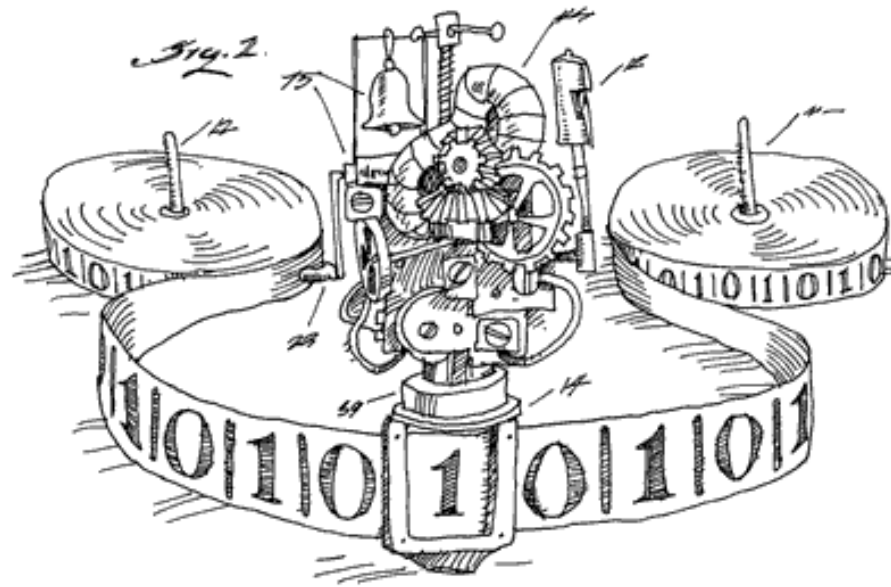


Turing Machines



Review: Representing problems

A **decision problem** can be thought of as

$$f : \Sigma^* \rightarrow \{\text{No}, \text{Yes}\}$$

or equivalently as a **language**

$$L \subseteq \Sigma^*$$

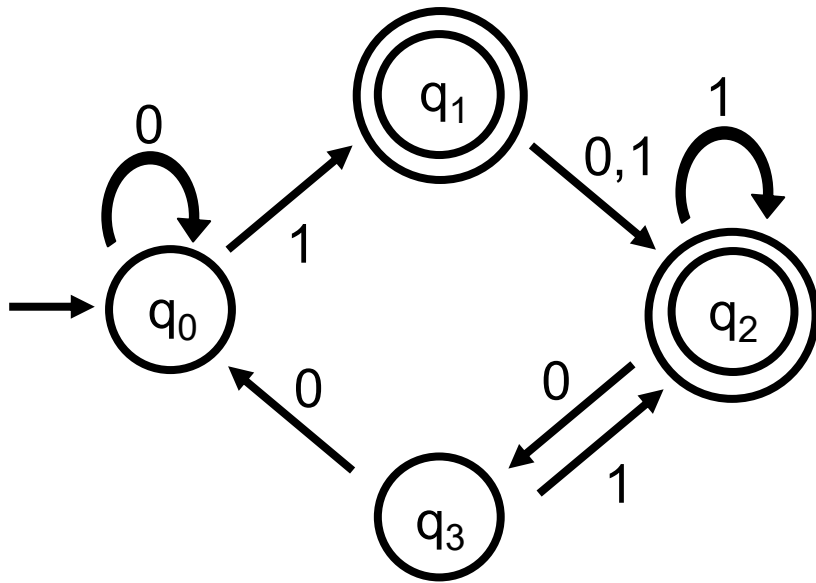
$$L = \{x \in \Sigma^* : f(x) = \text{Yes}\} \quad f(x) = \begin{cases} \text{Yes} & \text{if } x \in L \\ \text{No} & \text{if } x \notin L \end{cases}$$

E.g.: $L_{\text{PALINDROME}} = \{x \in \Sigma^* : x \text{ is a palindrome}\}$

Review: DFAs

A **deterministic finite automaton** is a 5-tuple:

$$M = (\Sigma, Q, q_0, F, \delta)$$



$$\Sigma = \{0,1\}$$

$$Q = \{q_0, q_1, q_2, q_3\}$$

q_0 is the start state

$$F = \{q_1, q_2\}$$

$$\delta : Q \times \Sigma \rightarrow Q$$

Review: Representing problems

Let M be a DFA, using alphabet Σ .

M **accepts** some strings in Σ^* and **rejects** the rest.

Definition: $L(M) = \{x \in \Sigma^* : M \text{ accepts } x\}$

Called the “language decided by M ”.

If L is a language,

we say M **decides** L if $L(M) = L$.

Impossibility Proof

Consider any DFA **M**. We show **M** does not decide $L = \{x^k y^k : k \geq 0\}$,

Let **s** = #states of **M**. (It's a finite number!)

Consider the (finite) input string $x^s y^s$.

As we process the string, the DFA switches to some different states q_i :

ε	x	xx	xxx	$xxxx$	$xxxxx$	$\dots$	x^s
q_{i_0}	q_{i_1}	q_{i_2}	q_{i_3}	q_{i_4}	q_{i_5}	$\dots$	q_{i_s}



There are **s states in M** (pigeonholes) and **s+1 states listed** (pigeons), so there must be *distinct* **i, j** $\leq s$ such that **state**(x^i) = **state**(x^j).

Claim. The two inputs $x^i y^i$ and $x^j y^i$ have the same final state. Why?

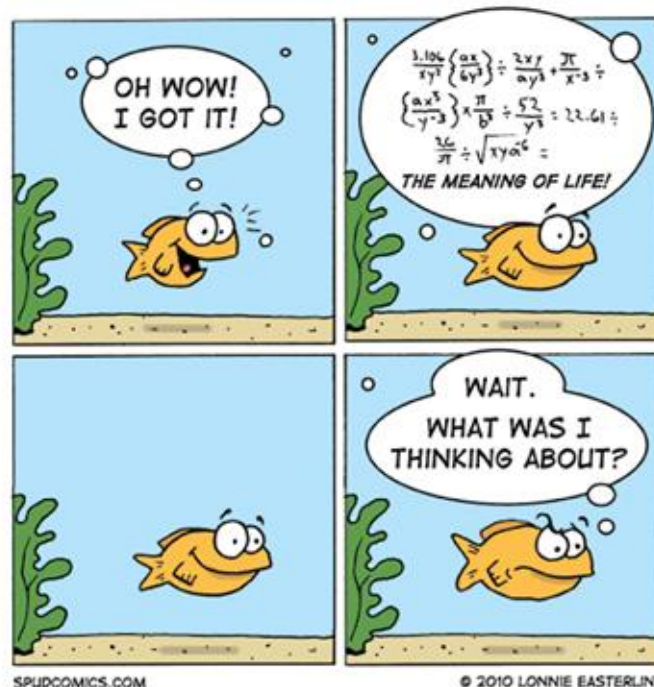
This means **M** must accept both or reject both!

But $x^i y^i \in L$ and $x^j y^i \notin L$, so **M** is wrong on one of these!

Conclusion: **M** does not decide L.

Intuitive reason why DFAs cannot decide many languages:

Fixed (finite) memory!



SPUDCOMICS.COM

© 2010 LONNIE EASTERLING

THE TRAGEDY OF A THREE SECOND MEMORY

A DFA models a computer with a fixed amount of memory
(think of the states as “memory”)

Real computers are only as powerful as DFAs

If you have only 1TB of storage, there must be a k (over 2.6 trillion *digits long*) where your computer will not give the right answer

Your laptop also cannot decide the language $L = \{x^k y^k : k \geq 0\}$.

But, in our definition of a “computer”, we don’t want to artificially constrain ourselves to a fixed amount of memory.

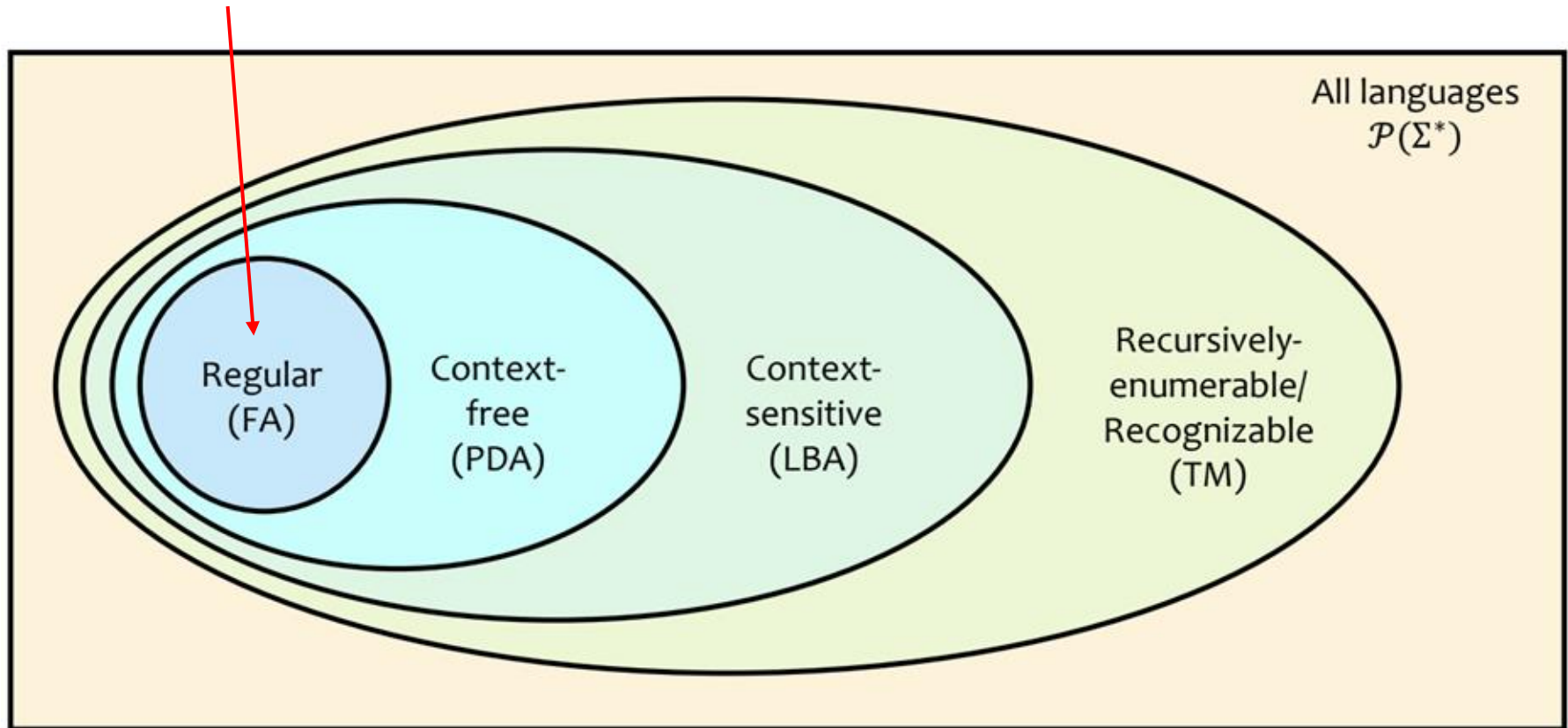
Instead, we want our definition of a “computer” to model the situation where you can always buy more memory if you need it.

That’s where **Turing Machines** come in.

The Chomsky Hierarchy (1956)



“Regular Language”: Language decidable by some DFA



More powerful “memory system” $\longrightarrow$

DFAs/regularity are widely used in practice

- Simple household devices (switches, garage doors, ...)
- Software for designing and checking the behavior of digital circuits
- “Lexers,” the first stage of compilers
- Flexible string search in text files (logs, latex, ...)
 - o **Warning:** “Regular exprs” in practice often *aren’t* regular!
- and much more...

Some History: Hilbert's 10th Problem (1900)

Consider a multivariate polynomial with integer coefficients, and set it to 0.

E.g. $4x^2y^3 - 2x^4z^5 + x^8 = 0$.



“Devise a *process according to which it can be determined in a finite number of operations* whether this has an integer solution.”

$x^2 - 4 = 0$ -> yes

$x^2 - 2 = 0$ -> no

$4x^2y^3 - 2x^4z^5 + x^8 = 0$ -> ????

This is asking for an **algorithm**.

Entscheidungsproblem (“decision problem”) (1928)

Given a statement in first-order logic.

Devise an “*effectively calculable procedure*”
that determines if it is valid.

“All kittens are cats, all cats are cute, therefore all kittens are cute” -> true

“All kittens are cats, all kittens are cute, therefore all cats are cute” -> false

“All even numbers greater than 2 are the sum of two primes” -> ?????

Again, this is asking for an **algorithm**.

Mathematicians began to think hard about a
formal definition for “**algorithm**”.



David Hilbert



Wilhelm Ackermann

Gödel (1934):

Discusses some ideas for mathematical definitions of computation procedures, but isn't confident what's a good definition.



Church (1936):

$$Y = \lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$$

Invents **lambda calculus**, essentially claims it should be the definition of “algorithm”.



Gödel, Post (1936):

Arguments that Church's claim is in not justified.



Meanwhile... in New Jersey... a certain British grad student, unaware of all these debates...

Alan Turing (1936, age 22):

Describes a new model of computation,
now known as the **Turing Machine**TM



Gödel, Kleene, and even Church:

“Um, he nailed it. Game over, ‘algorithm’ defined.”

1937: Turing proves **TM's $\equiv$ lambda calculus**

In Theory Celebrates the Turing Centennial

Posted on [May 27, 2012](#)

This year there are several festivities on the occasion of [Turing](#)'s 100th birthday, which is next month, including the celebration that took place in [Princeton](#) a couple of weeks ago, the ACM-organized [event](#) next month, several events at the Newton institute in the UK, and several other events [all over the world](#). Most of Turing centennial initiatives have taken the form of lectures and articles describing how far we have (or not) gone since Turing's time in our understanding of computation.

Certainly, a big part of Turing's life was being gay, which was not exceptional among [leading British mathematicians](#) of his time, or even among [founding fathers of theoretical computer science](#), although his way of being "out" was (like his research) much ahead of its time.

⋮

So I have invited a number of gay and lesbian colleagues to write guest posts talking about how things have been for them, and so far half a dozen have tentatively accepted. Their posts will appear next month which, besides being Turing's centennial month, also happens to be the anniversary of the [Stonewall riots](#).



Luca Trevisan

<https://lucatrevisan.wordpress.com/tag/turing-centennial/>

Church–Turing Thesis

“Any natural notion of being ‘algorithmically computable’ is captured by Turing Machines.”

This is not a theorem.

Is it... *... a hypothesis?*

... a definition?

... a philosophical statement?

Well, in any case, everyone seems to believe it.

Anything that can be computed by Python, C++, a quantum computer, LaTeX, pseudocode, etc. can be computed by a Turing Machine.

Entscheidungsproblem: Devise an **algorithm** (Turing Machine) that, given a statement in first-order logic, determines if it's valid.



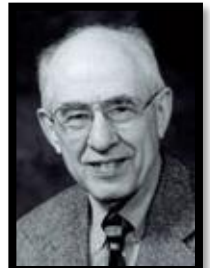
There is no such algorithm! (1936)



Hilbert's 10th Problem: Devise an **algorithm** (Turing Machine) that, given a multivariate polynomial with integer coefficients, determines if it has an integer root.

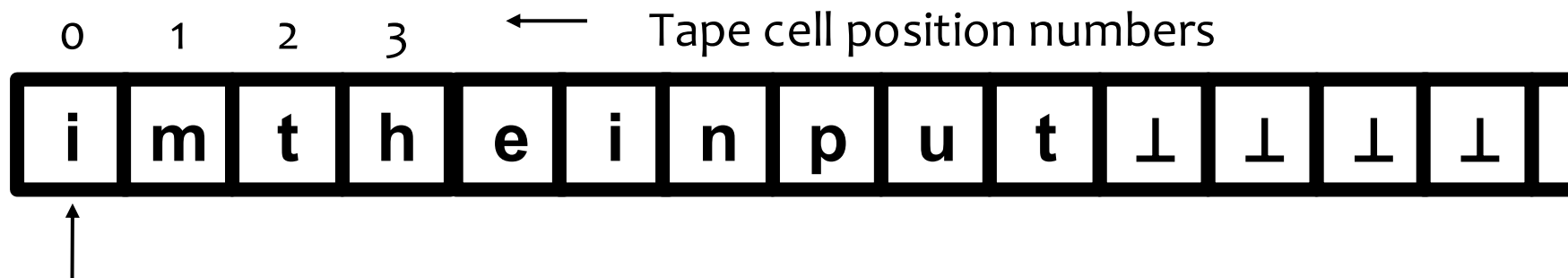


There is no such
algorithm! (1970)



What is a Turing Machine?

Memory of a Turing Machine: an infinite “tape” of cells.



The input from Σ^* is written starting at cell 0.

All other cells contain $\perp$ (blank).

In general, cells contain symbols from a (finite) **tape alphabet** Γ , which must contain everything in Σ , $\perp$, and may have other symbols.

There's a tape pointer (“**head**”), initially at position 0.

What is a Turing Machine?

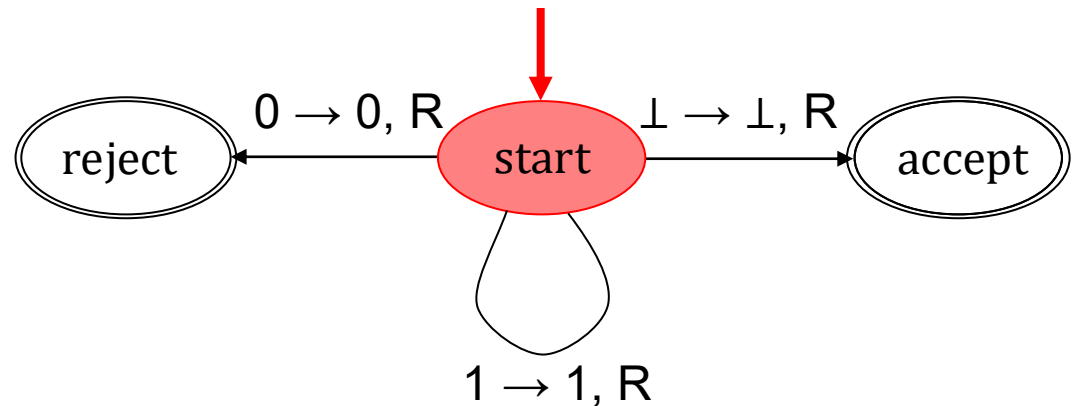
Like a DFA, except each transition also specifies:

- what we *write* to the tape
- whether the head moves *left* (L) or *right* (R)

Specifically: “**a** → **b**, **R**” means:

if the current tape cell is **a**:

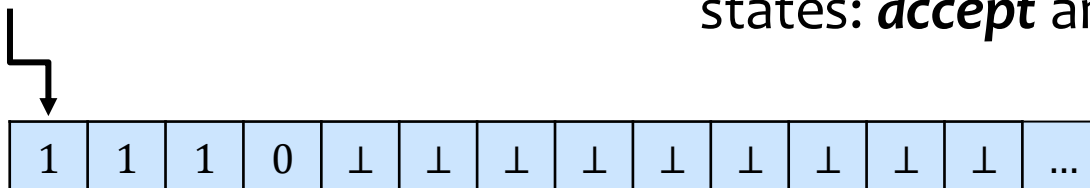
1. write **b**
2. move **right**



Assume that if TM encounters “move left” with head at position 0, the head doesn’t move



INFINITE
TAPE



There are two special “termination” states: **accept** and **reject**.

What is a Turing Machine?

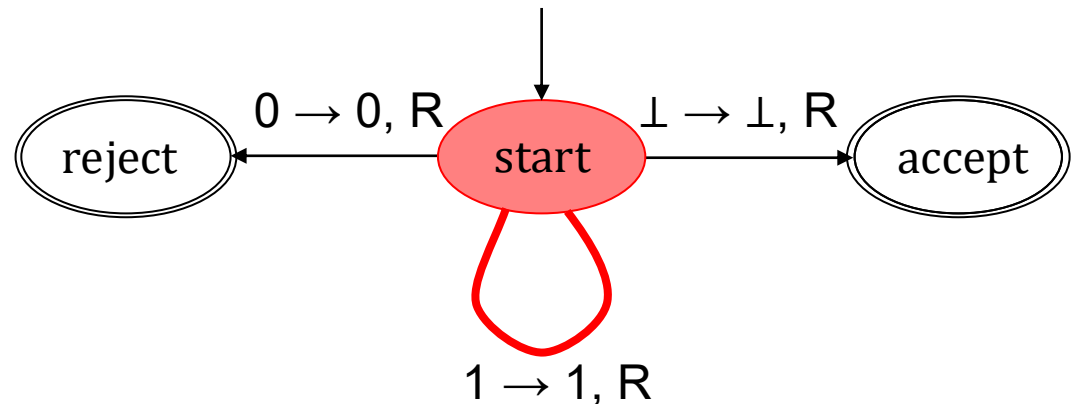
Like a DFA, except each transition also specifies:

- what we *write* to the tape
- whether the head moves *left* (L) or *right* (R)

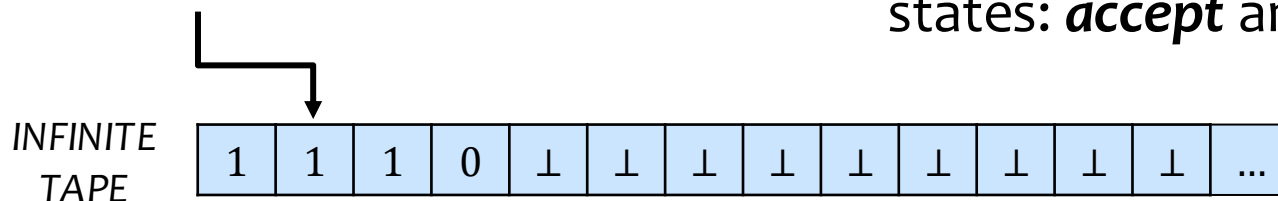
Specifically: “**a** → **b**, **R**” means:

if the current tape cell is **a**:

1. write **b**
2. move **right**



There are two special “termination” states: **accept** and **reject**.



What is a Turing Machine?

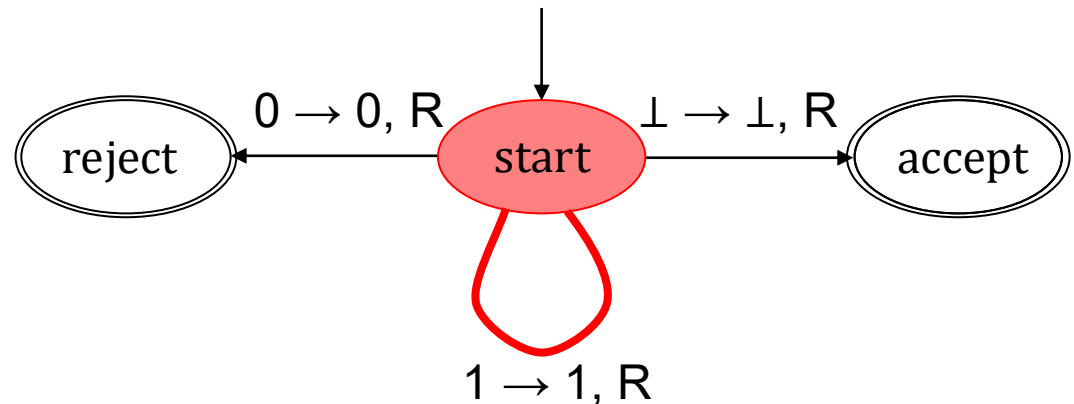
Like a DFA, except each transition also specifies:

- what we *write* to the tape
- whether the head moves *left* (L) or *right* (R)

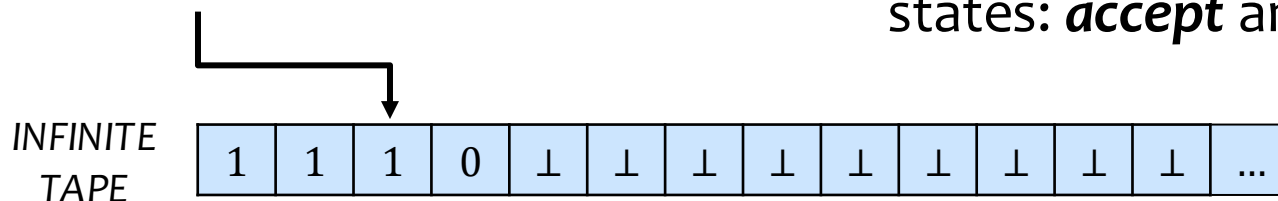
Specifically: “**a** → **b**, **R**” means:

if the current tape cell is **a**:

1. write **b**
2. move **right**



There are two special “termination” states: **accept** and **reject**.



What is a Turing Machine?

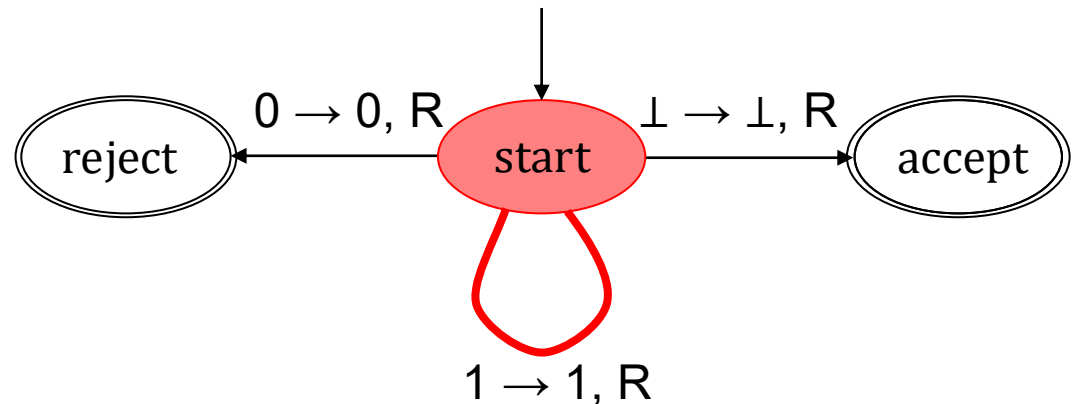
Like a DFA, except each transition also specifies:

- what we *write* to the tape
- whether the head moves *left* (L) or *right* (R)

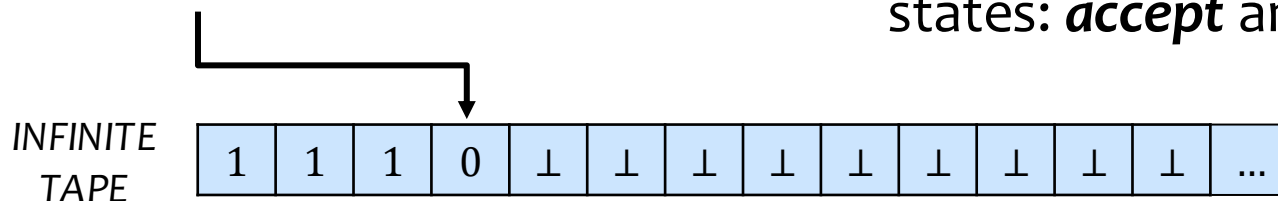
Specifically: “**a** → **b**, **R**” means:

if the current tape cell is **a**:

1. write **b**
2. move **right**



There are two special “termination” states: **accept** and **reject**.



What is a Turing Machine?

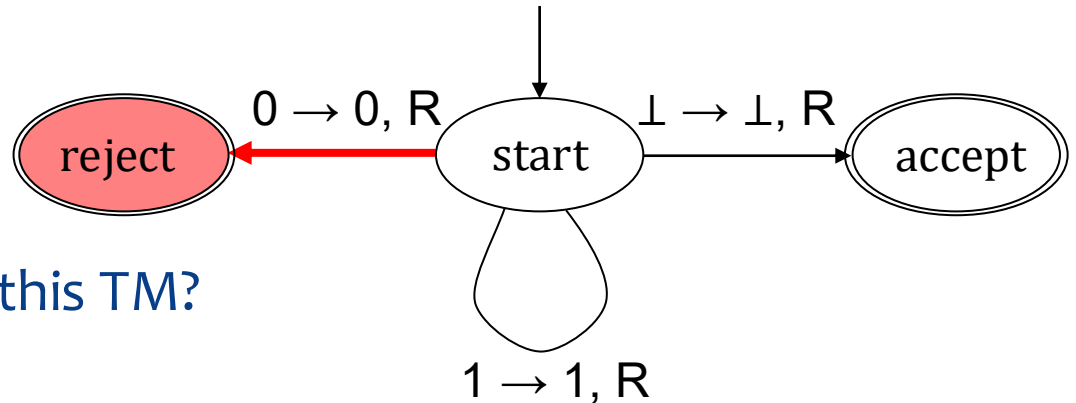
Like a DFA, except each transition also specifies:

- what we *write* to the tape
- whether the head moves *left* (L) or *right* (R)

Specifically: “ **$a \rightarrow b, R$** ” means:

if the current tape cell is **a**:

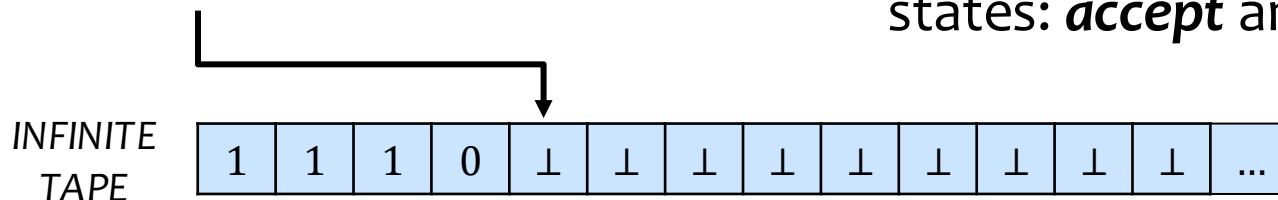
1. write **b**
2. move **right**



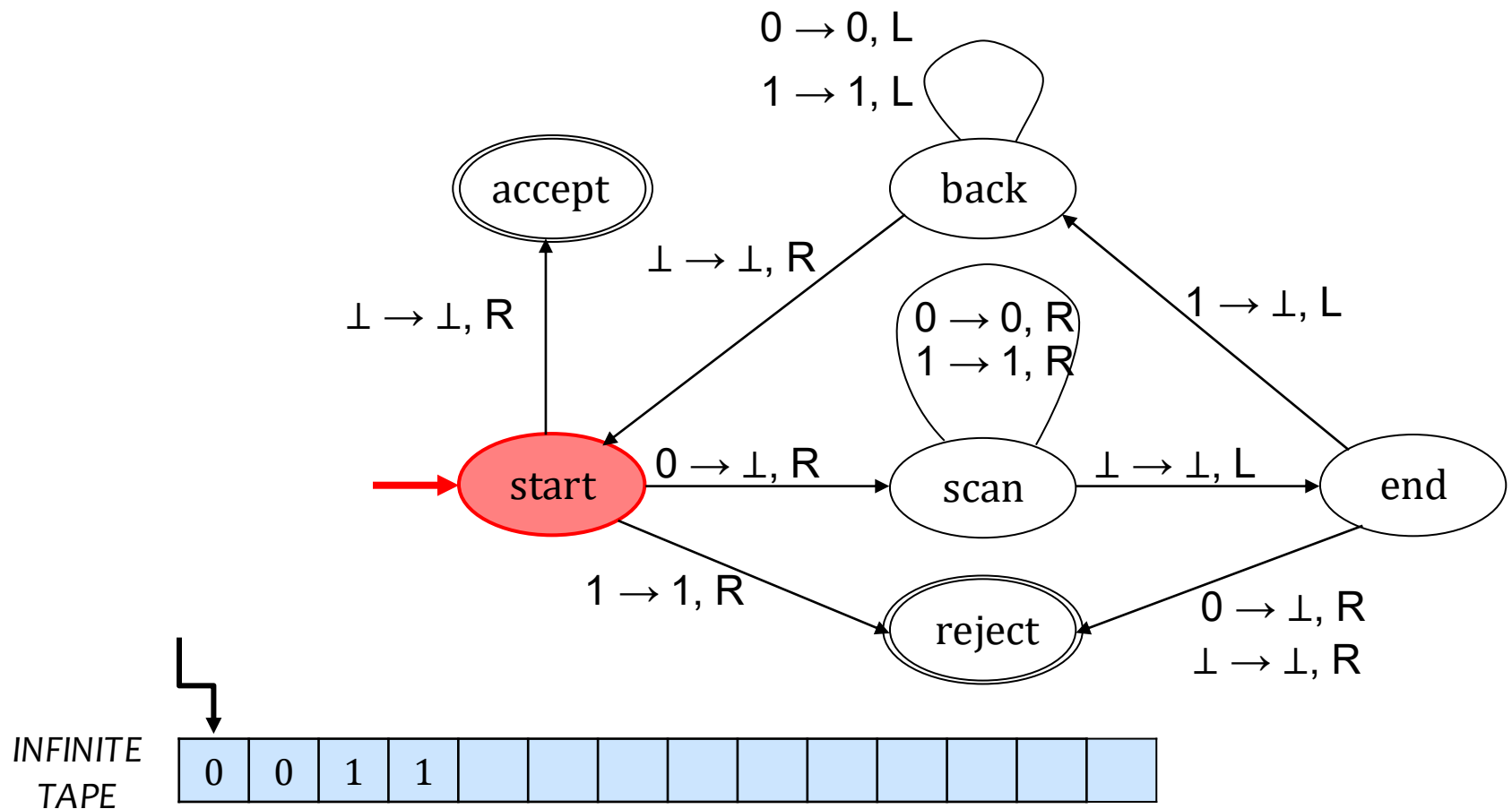
Q: Set of inputs accepted by this TM?

A: 1*

There are two special “termination” states: ***accept*** and ***reject***.

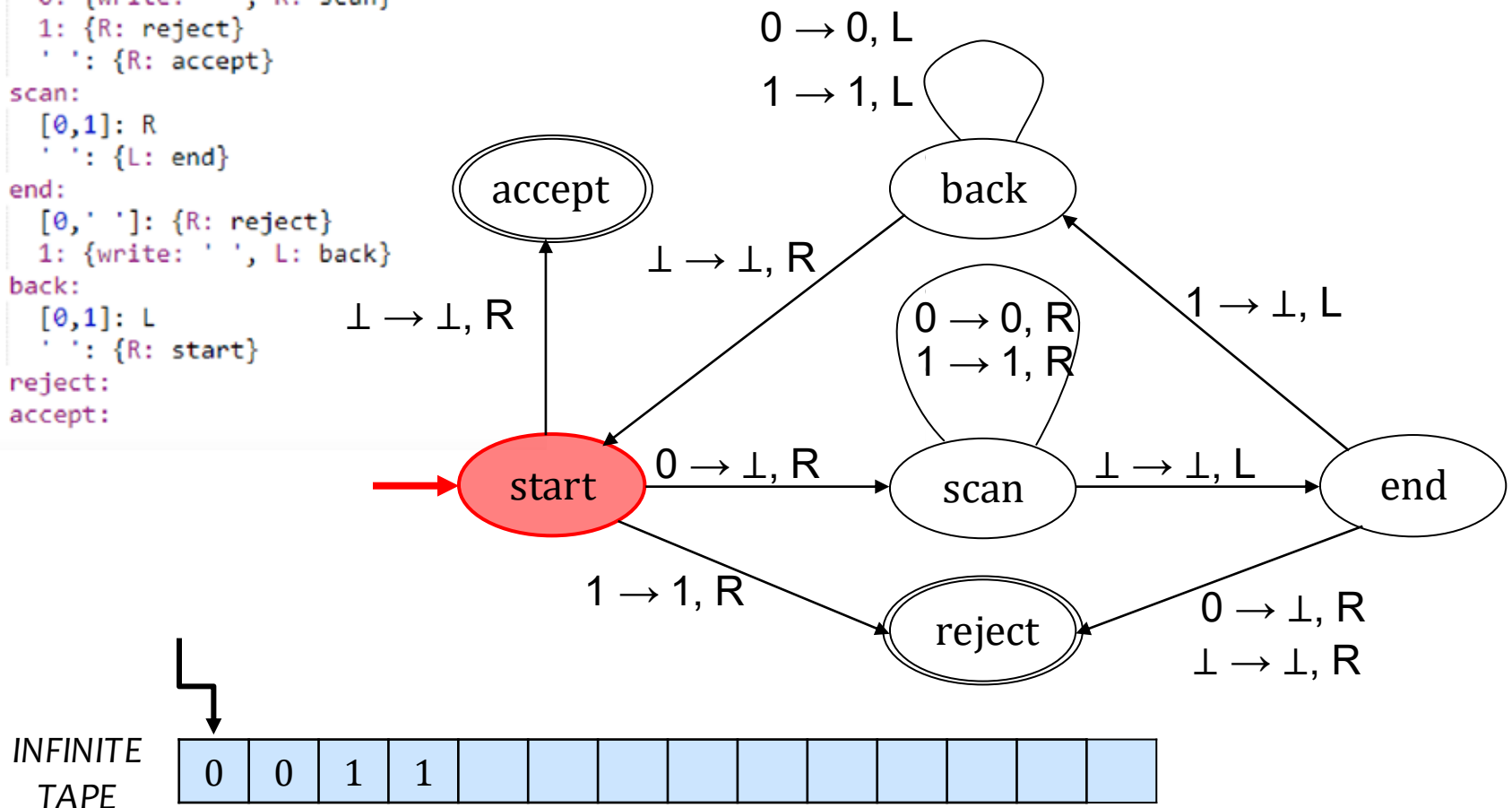


What is a Turing Machine?

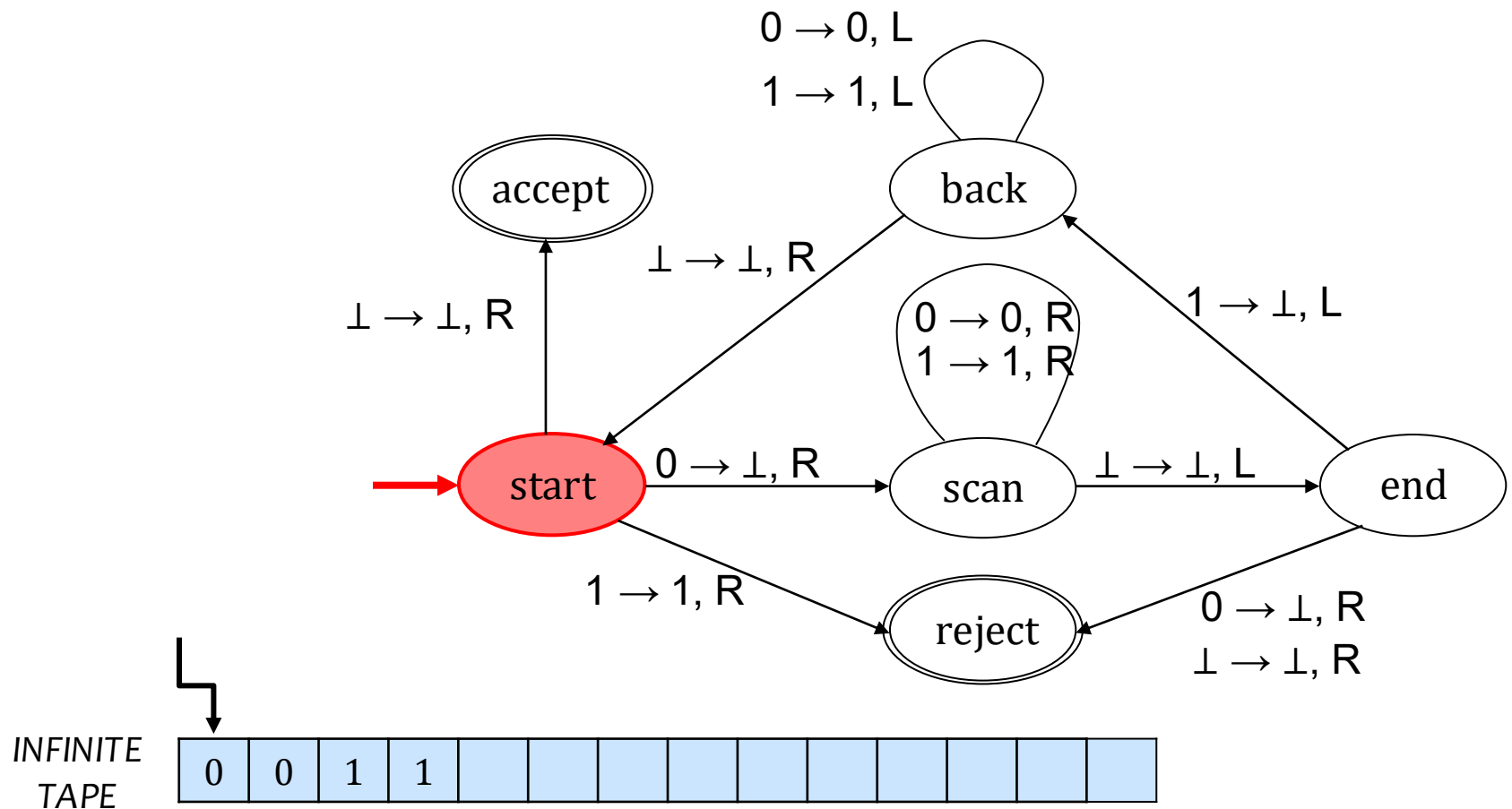


A useful tool: turingmachine.io

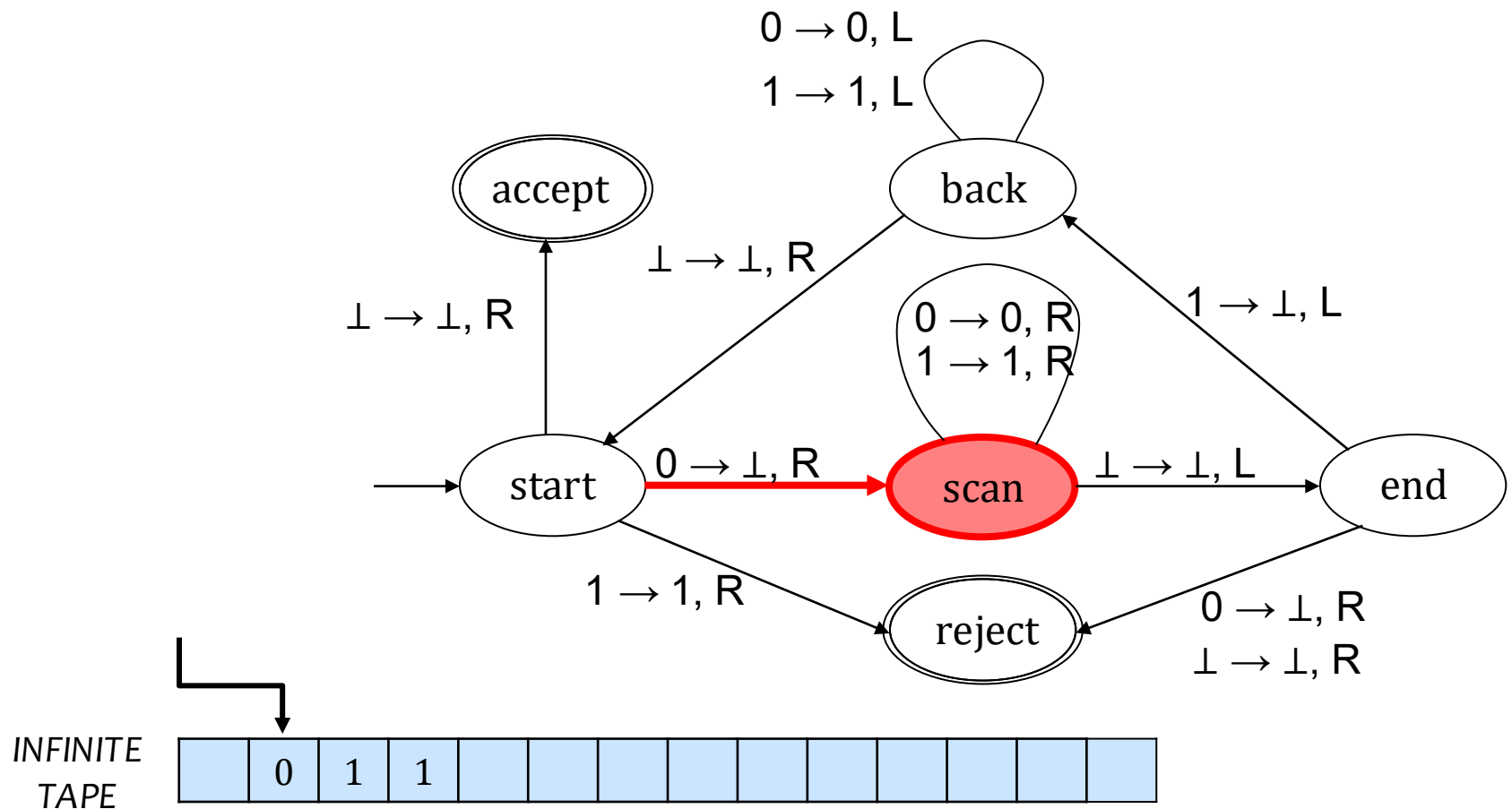
```
3 input: '0011' # try '', '0', '011', '010', '11', '0101'
4 blank: ''
5 # Blank out the first 0 and last 1 on each pass.
6 # When there are only blanks, all 0's have matched to all 1's.
7 start state: start
8 table:
9   start:
10    0: {write: '', R: scan}
11    1: {R: reject}
12    ' ': {R: accept}
13   scan:
14    [0,1]: R
15    ' ': {L: end}
16   end:
17    [0, ' ']: {R: reject}
18    1: {write: '', L: back}
19   back:
20    [0,1]: L
21    ' ': {R: start}
22 reject:
23 accept:
```

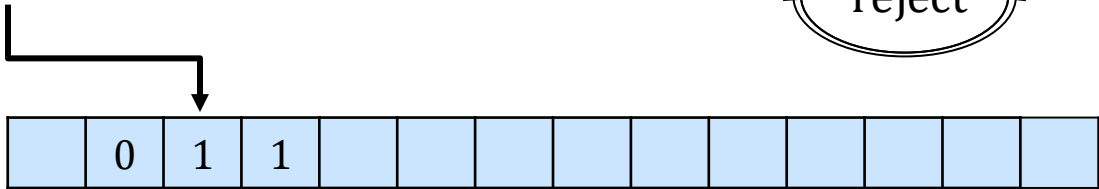


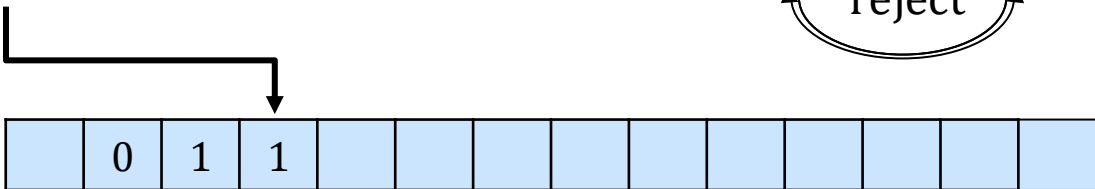
What is a Turing Machine?

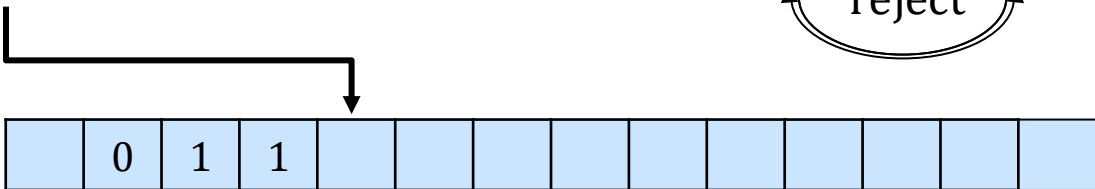


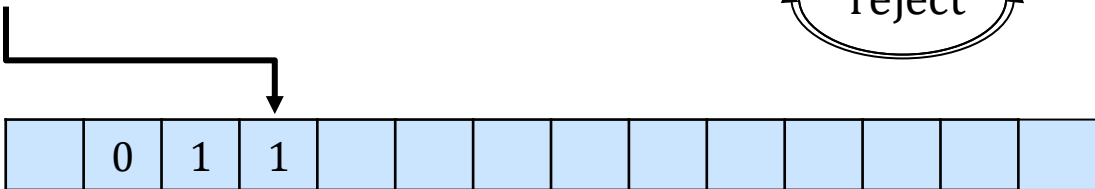
What is a Turing Machine?





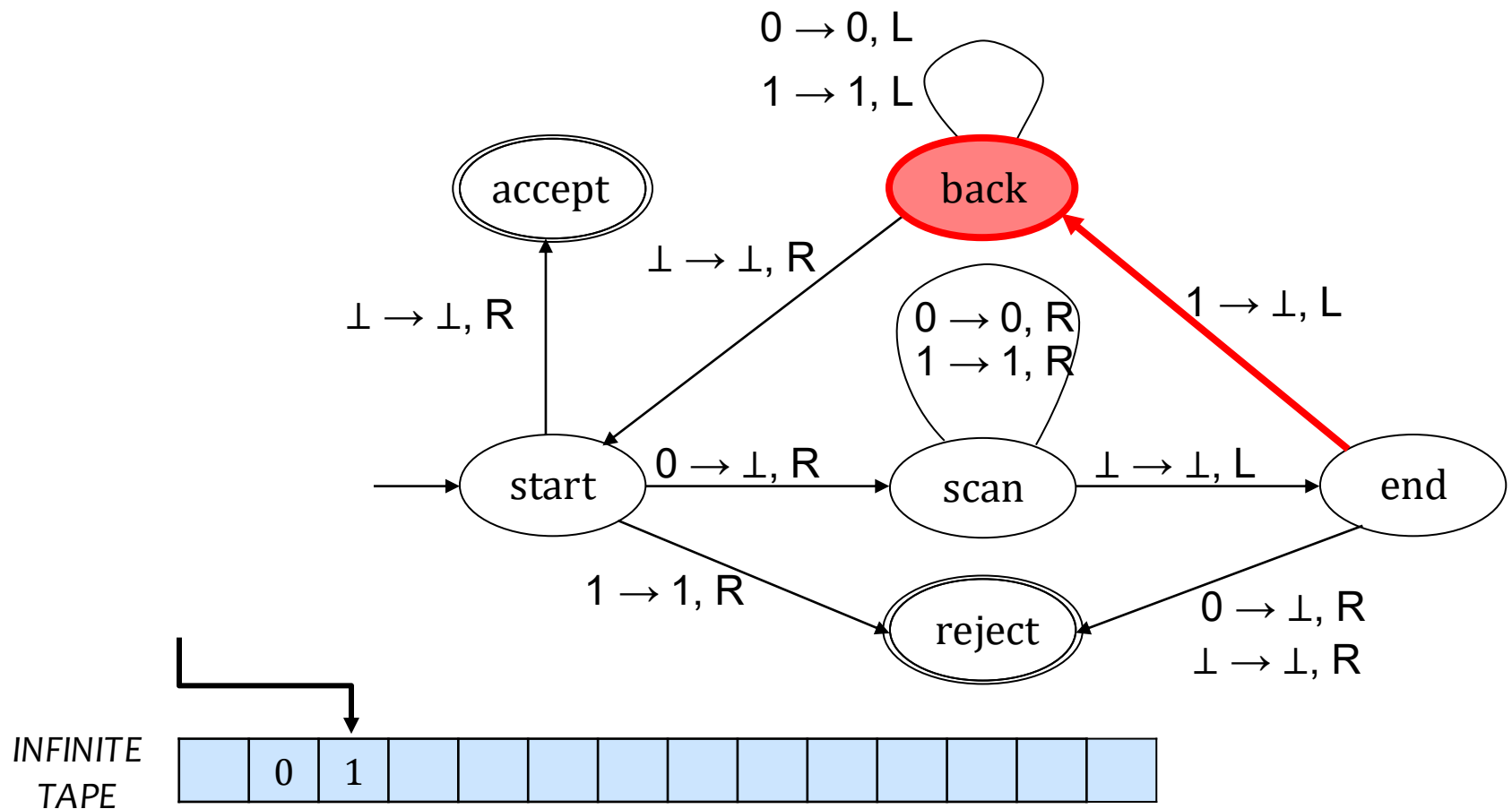






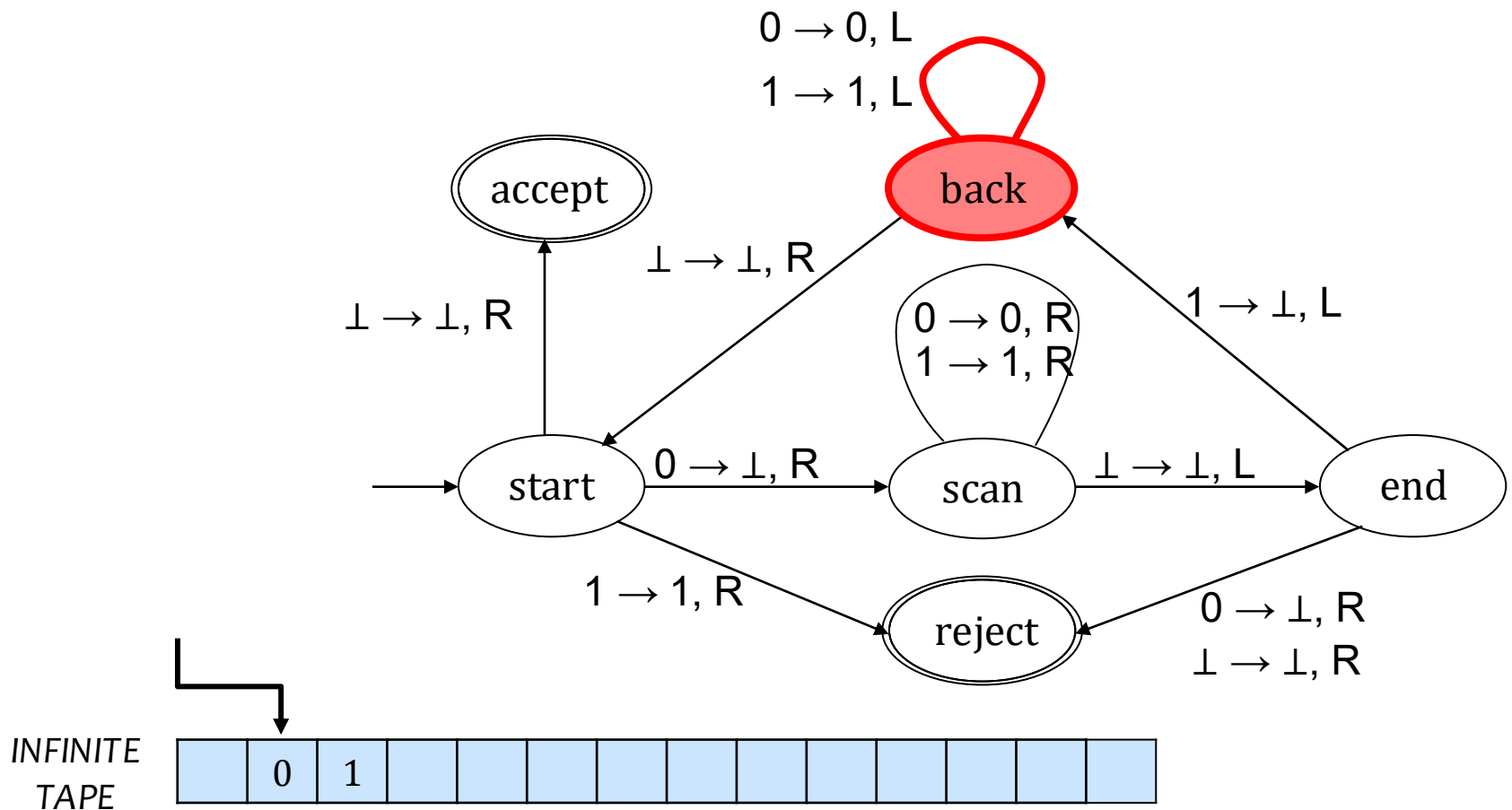
What is a Turing Machine?

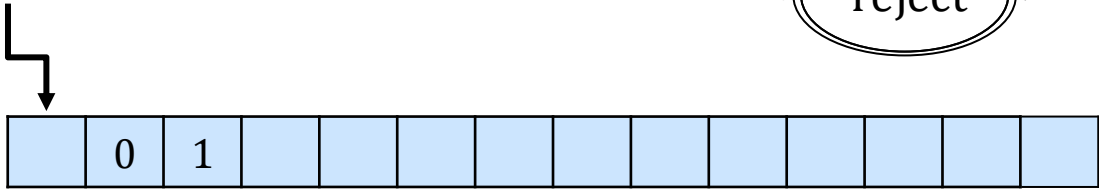
EXAMPLE



What is a Turing Machine?

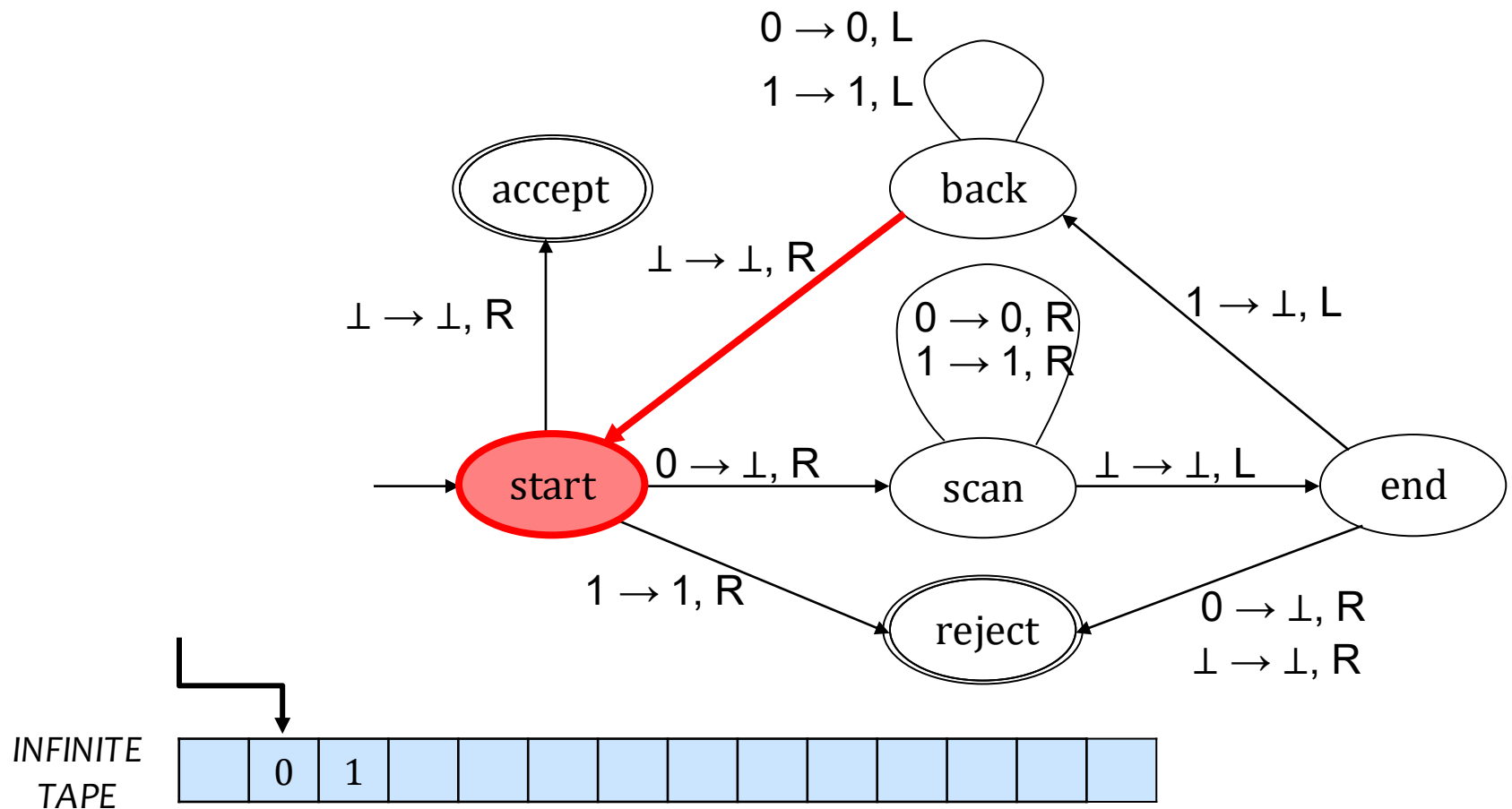
AN EXAMPLE





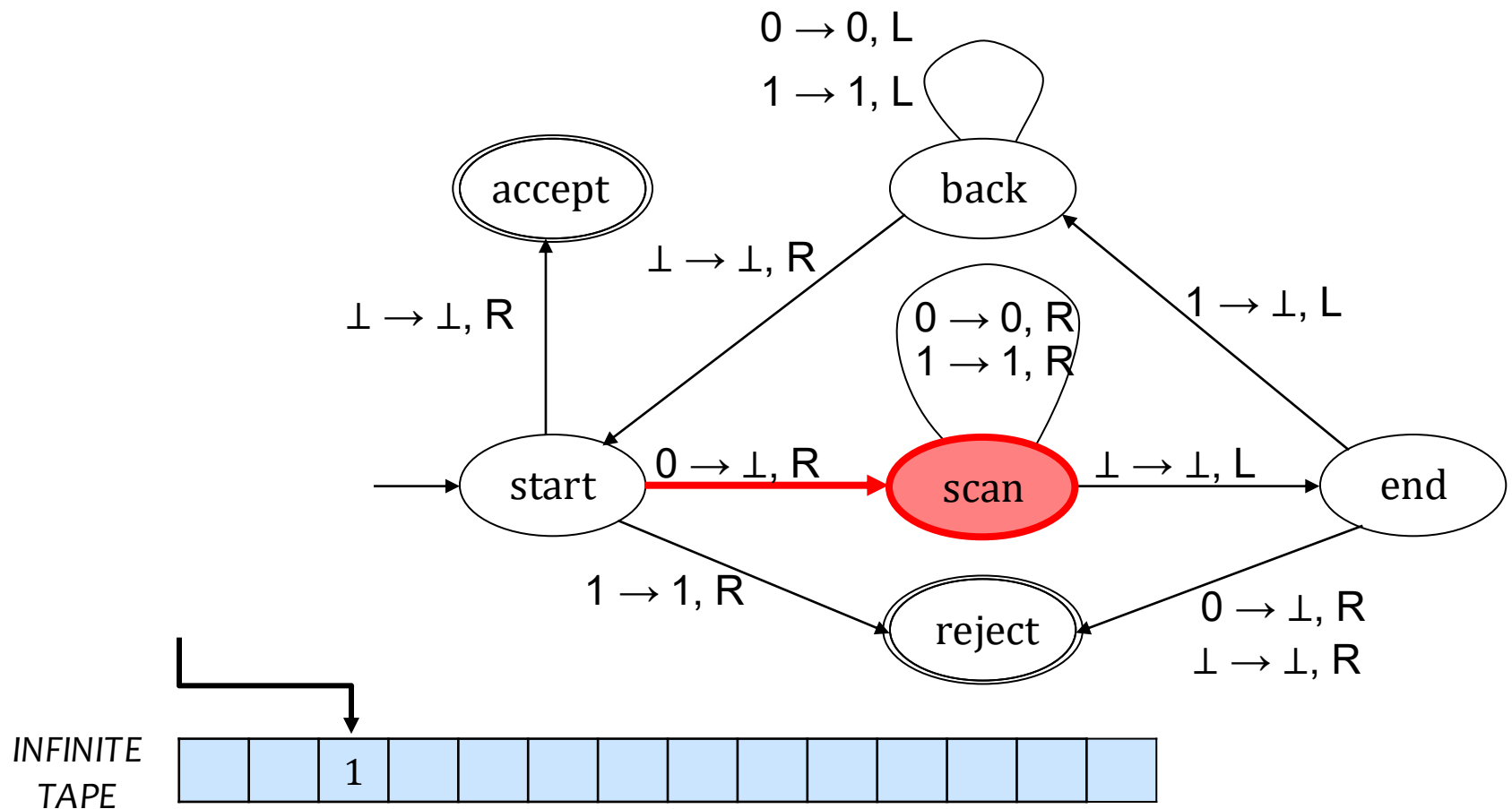
What is a Turing Machine?

TM EXAMPLE



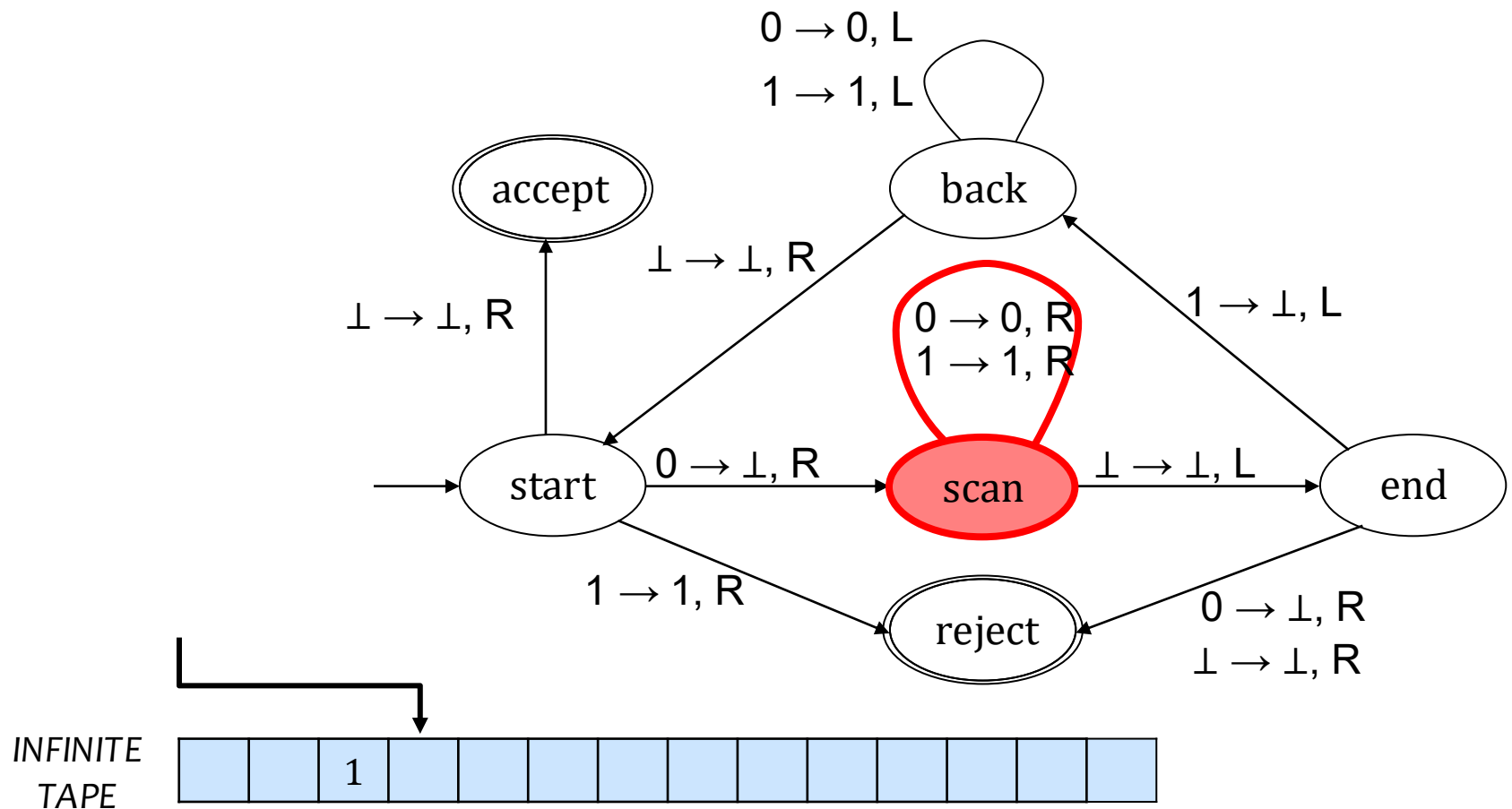
What is a Turing Machine?

AN EXAMPLE



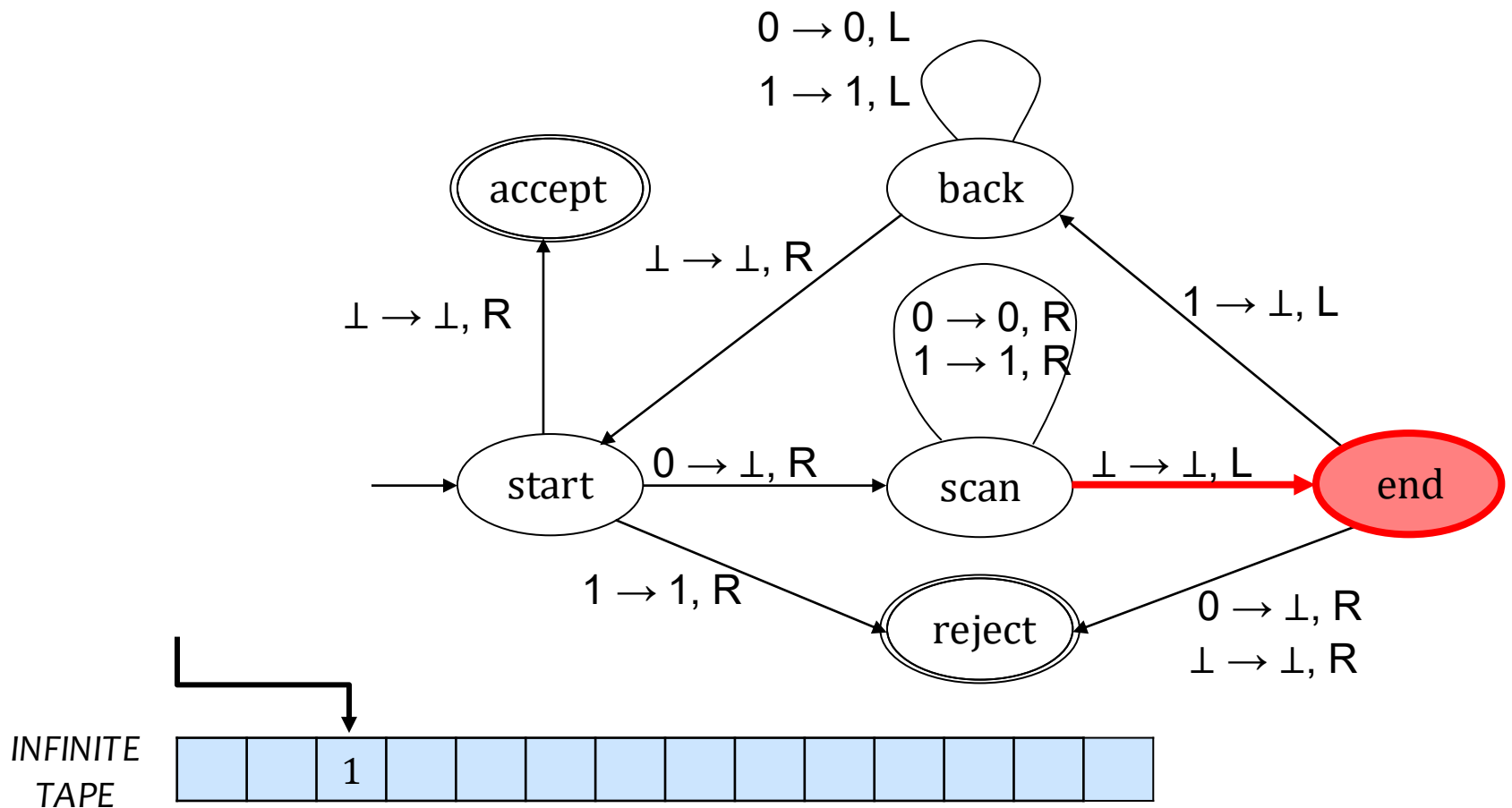
What is a Turing Machine?

AN EXAMPLE



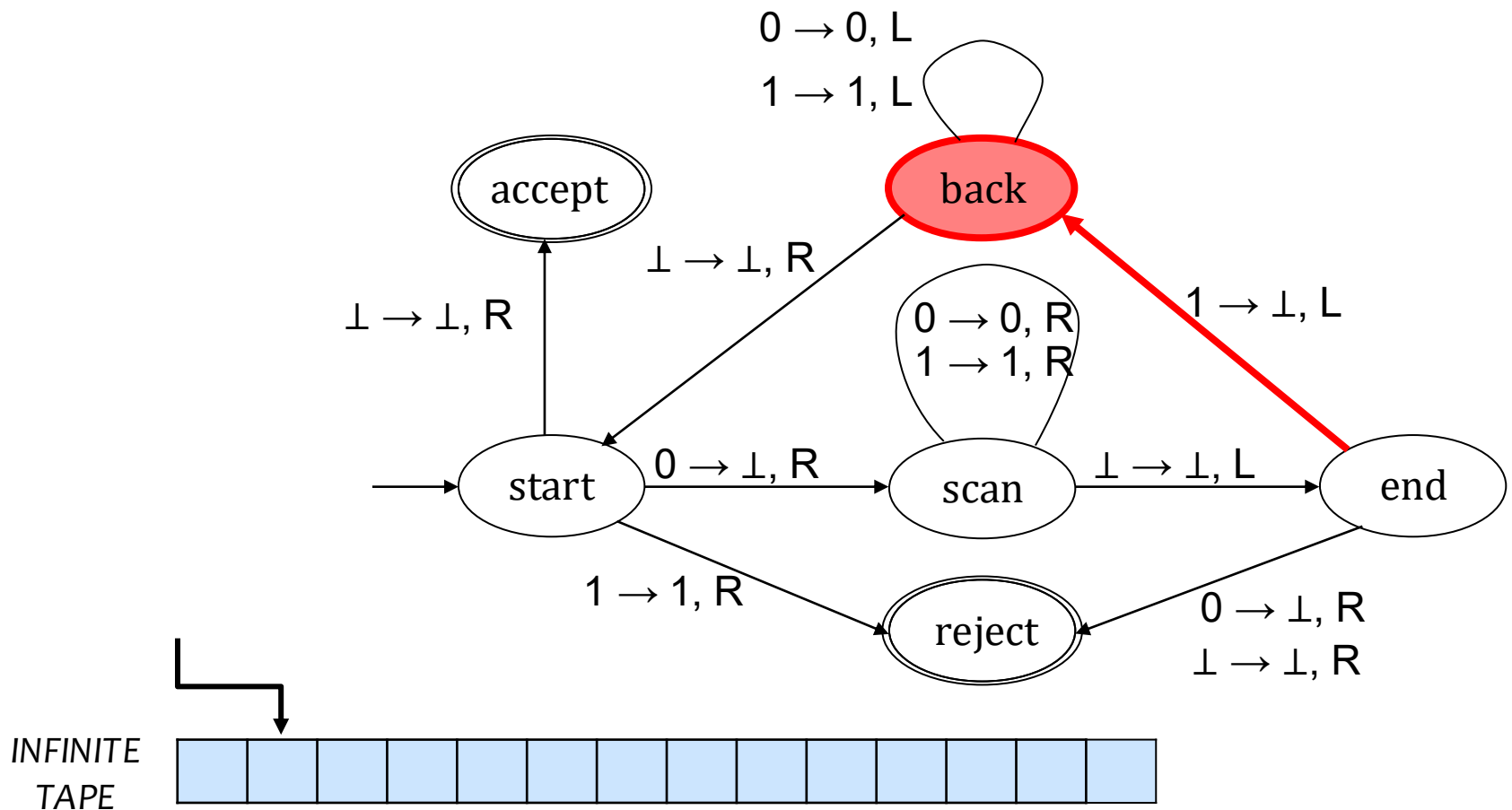
What is a Turing Machine?

AN EXAMPLE



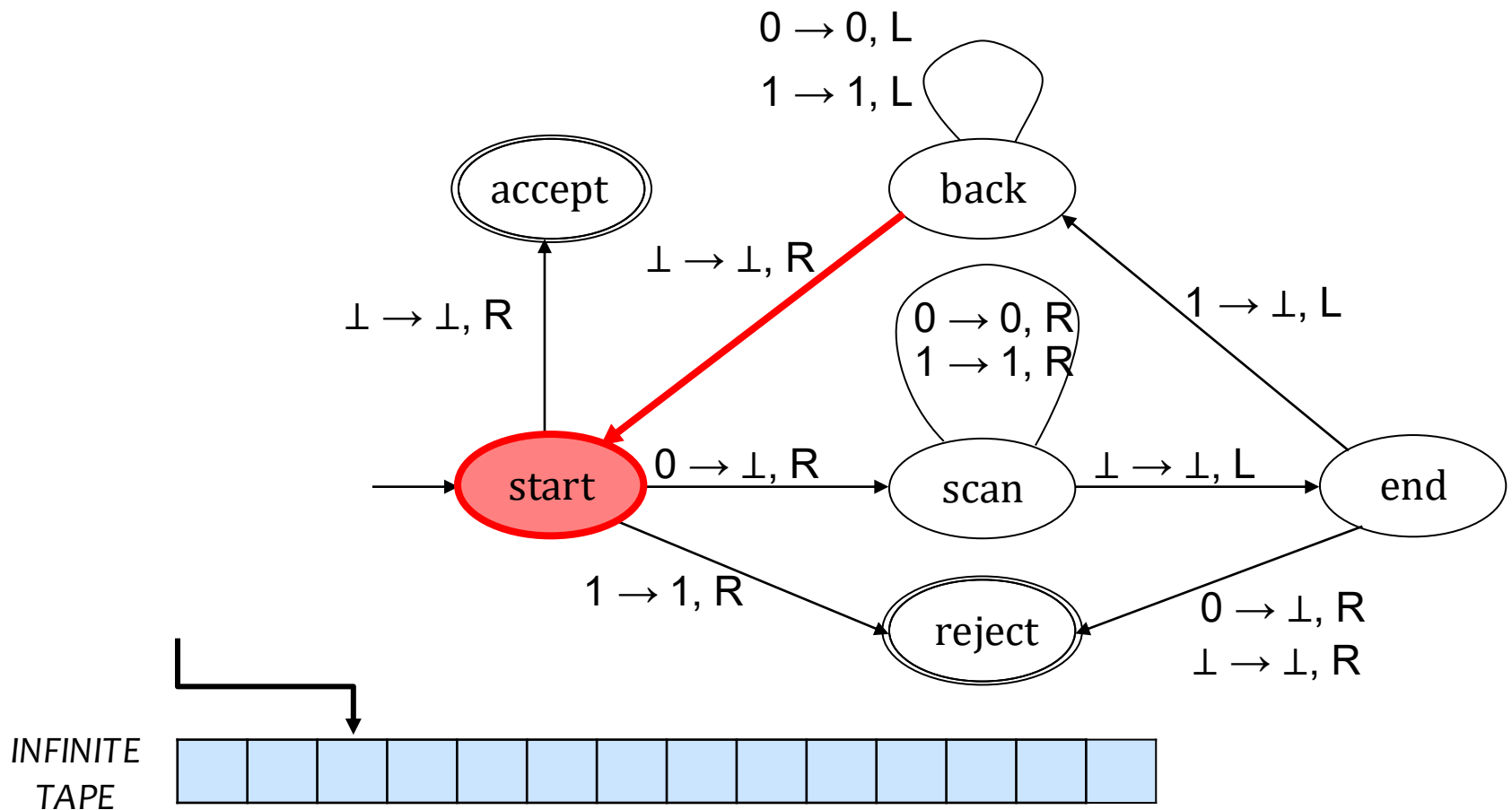
What is a Turing Machine?

EXAMPLE



What is a Turing Machine?

TM EXAMPLE

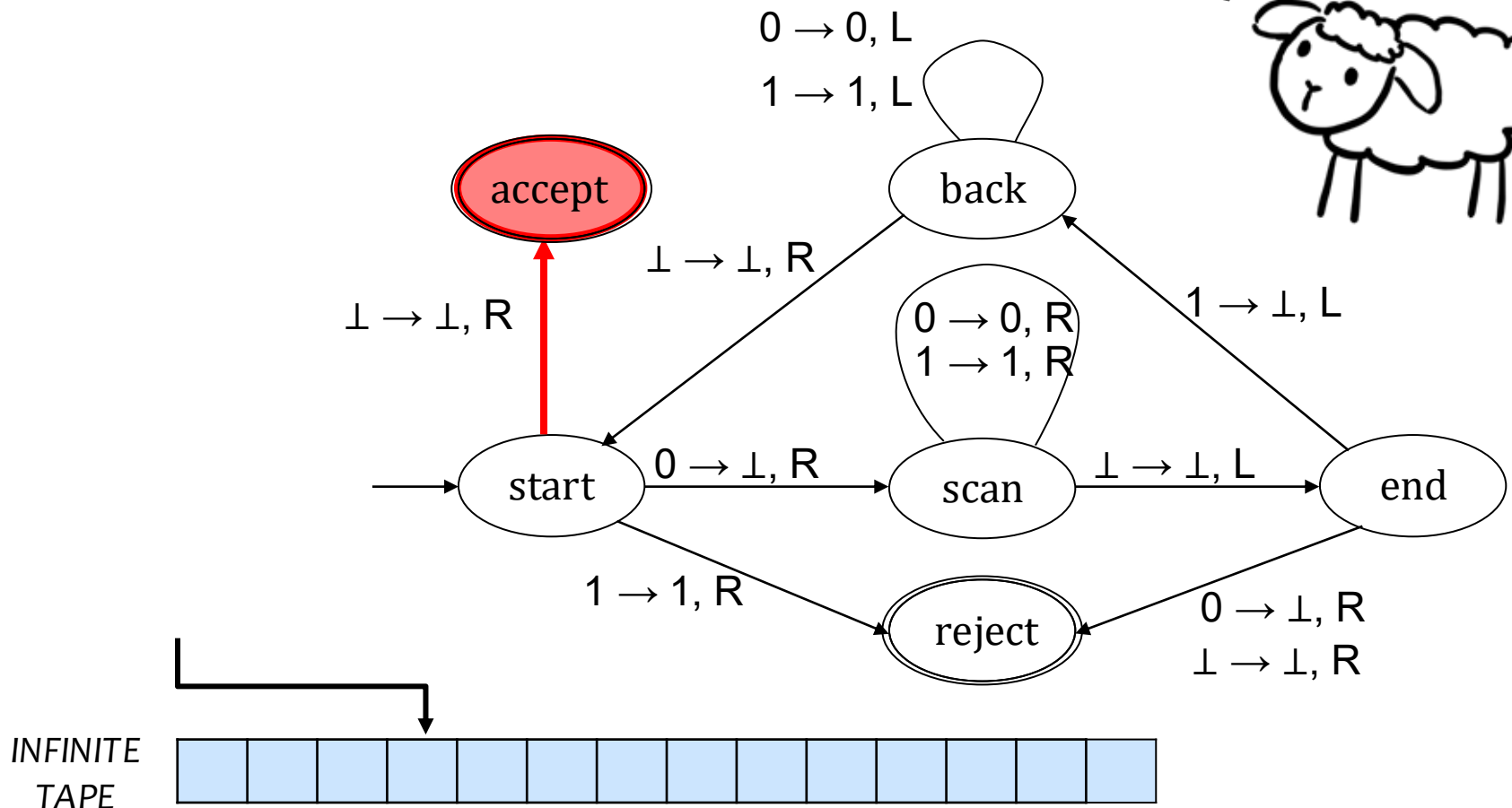


What is a Turing Machine?

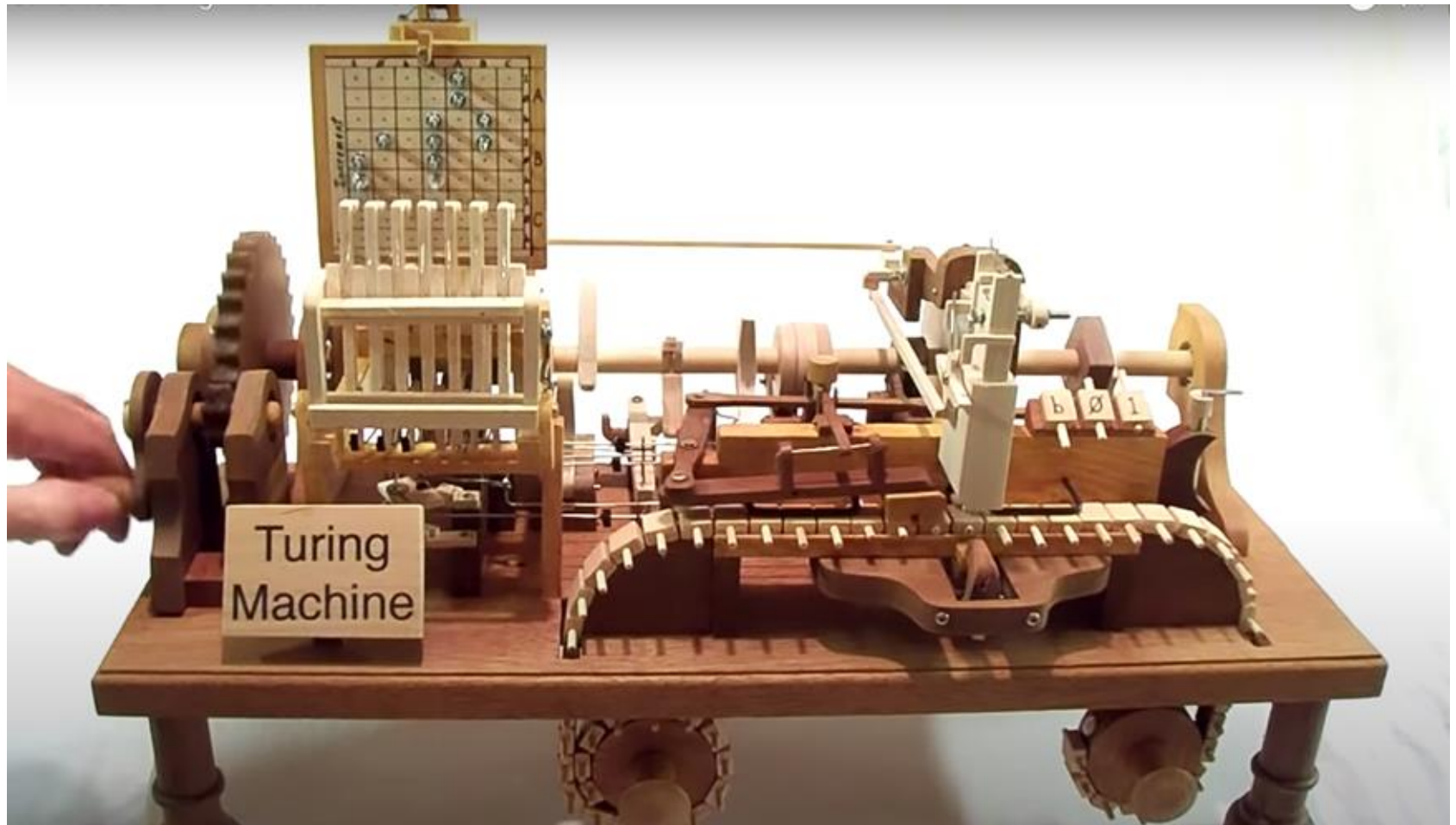
Q: Set of inputs accepted by this TM?

A: $\{0^k 1^k : k \geq 0\}$

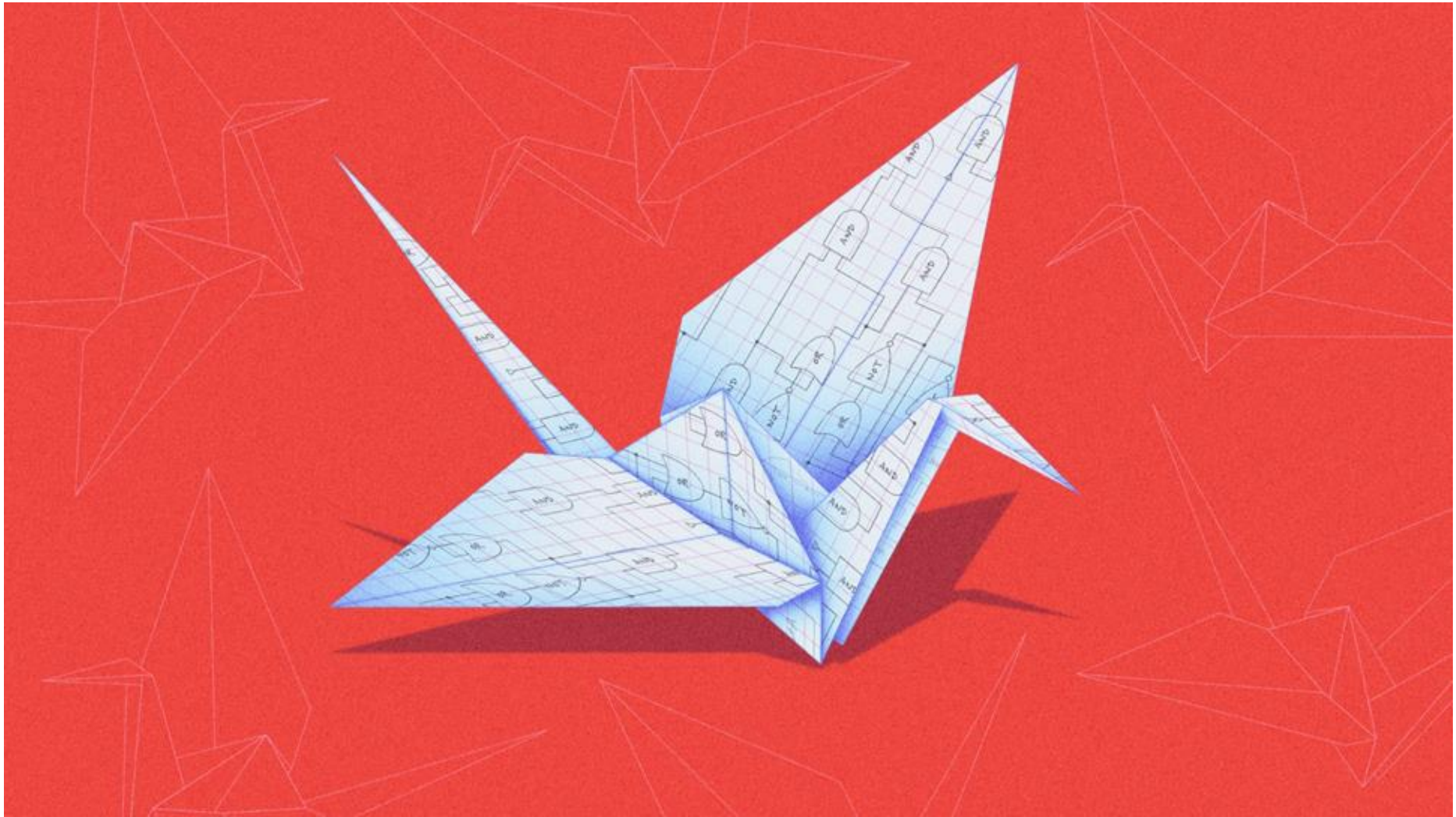
Whenever I program a Turing Machine I need to take a nap to recover



A Mechanical Turing Machine



Another Mechanical Turing Machine



["How to Build an Origami Computer"](#), Quanta Magazine, 30 Jan 2024

Another Mechanical Turing Machine



[“Magic: The Gathering is Turing Complete”](#), arXiv:1904.09828

[“I Built a COMPUTER in Magic: The Gathering”](https://youtu.be/pdmODVYPDLA), <https://youtu.be/pdmODVYPDLA>

Formal definition of a Turing Machine

A **Turing Machine** is a 7-tuple:

$$M = (\Sigma, \Gamma, Q, q_{\text{start}}, q_{\text{acc}}, q_{\text{rej}}, \delta)$$

All of these
sets are
finite



- Σ is the (finite) input alphabet,
- Γ is the (finite) tape alphabet,
 $\Sigma \subseteq \Gamma$, Contains blank symbol $\perp \in \Gamma$, $\perp \notin \Sigma$
- Q is the *finite* set of states,
 - $q_{\text{start}} \in Q$ is the *initial* state,
 - $q_{\text{acc}}, q_{\text{rej}} \in Q$ are the (distinct) *accept* and *reject* states,
 $F = \{q_{\text{acc}}, q_{\text{rej}}\}$
- δ is the *transition function*.

$$\delta: \underbrace{(Q \setminus F)}_{\text{Non-terminal state}} \times \underbrace{\Gamma}_{\text{Cell contents}} \rightarrow \underbrace{Q}_{\text{Next state}} \times \underbrace{\Gamma}_{\text{Symbol to write}} \times \underbrace{\{L, R\}}_{\text{Where to move the head}}$$

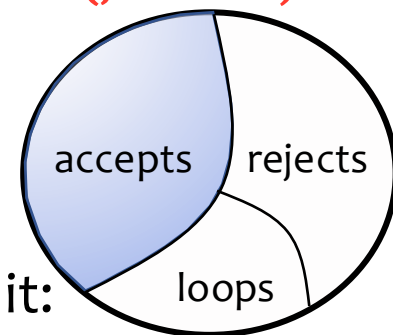
The Language of a TM

Question: What are the possible outcomes of a TM **M** on an input x ?

Answer: $M(x)$ either (i) accepts, (ii) rejects, or (iii) it “**loops**” (forever)

The language of a TM is the set of strings it accepts:

$$L(M) = \{x : M \text{ accepts } x\}$$



Definition: A Turing Machine **M** **recognizes** a language L if it:

1. accepts every string in L , and
2. rejects OR loops for every string not in L .

Equivalently, $L(M) = L$

Definition: A Turing Machine **M** **decides** a language L if it:

1. accepts every string in L , and
2. rejects every string not in L (and does not loop on any input)

Equivalently, $L(M) = L$,
and M always halts

A language L is **decidable** if there is a TM that decides L .

Otherwise L is **undecidable**.

Fact: If a language **L** is decidable, then the **complement of L** is also decidable. Why?

Just swap q_{acc} and q_{rej} !

Fact: If languages **L₁** and **L₂** are both decidable, then **$L_1 \cap L_2$** is decidable. Why?

Let M_{L_1} decide L_1 and M_{L_2} decide L_2
Construct a TM M:
 Run M_{L_1} and M_{L_2} on your input
 If both accept, accept
 else, reject
Claim: M decides $L_1 \cap L_2$

(a similar proof can be made for **$L_1 \cup L_2$**)

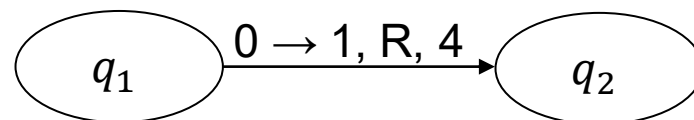
This doesn't look like a Turing Machine!
Because pseudocode is Turing Complete, we could convert this into a (very complicated) Turing Machine

What if M_{L_1} or M_{L_2} doesn't halt?
They "decide" L_1 and L_2 , so they must halt on all inputs!

Tape Jumping TM Equivalence Proof

Claim: A TM that allows the transitions to move the head any distance is exactly as powerful as a standard TM

- E.g. $\delta(q_1, 0) = (q_2, 1, R, 4)$



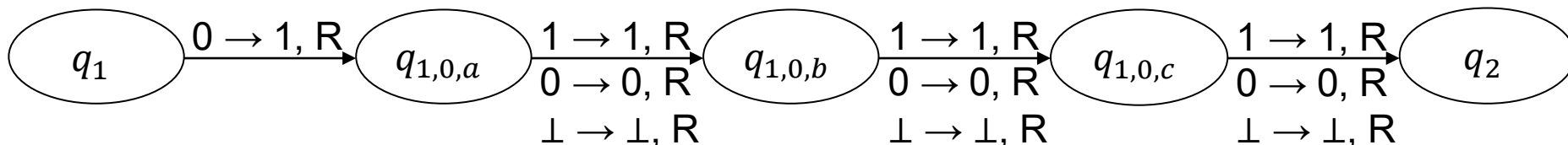
- “If in state q_1 and read 0, transition to q_2 , write 1, move 4 spaces right”

Anything a standard TM can do, a tape jumping TM can also do

- Have every transition move 1. Just don’t jump.

Anything a tape jumping TM can do, a standard TM can also do

- Insert extra states for each jump
- Note: each jump is fixed and finite, so we still have finite states



New machine can simulate standard TM

Very simple (trivial)

New machine is at least as powerful as a Turing Machine

“C++ can compute anything!”

Claim: A TM that can jump
distance is exponentially larger

Standard TM can simulate new machine

Tricky, requires cleverness

Backs up the Church–Turing Thesis

Allows us to use this new power in future machines and still call them Turing Machines

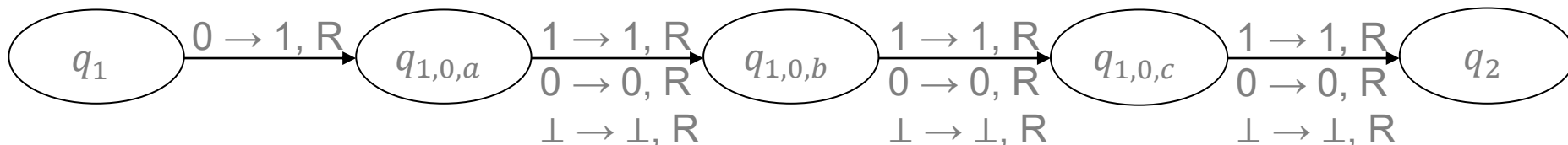
“A Turing Machine can run Doom!”

Anything a standard TM can do, a tape jumping TM can also do

- Have every transition move 1. Just don't jump.

Anything a tape jumping TM can do, a standard TM can also do

- Insert extra states for each jump
- Note: each jump is fixed and finite, so we still have finite states



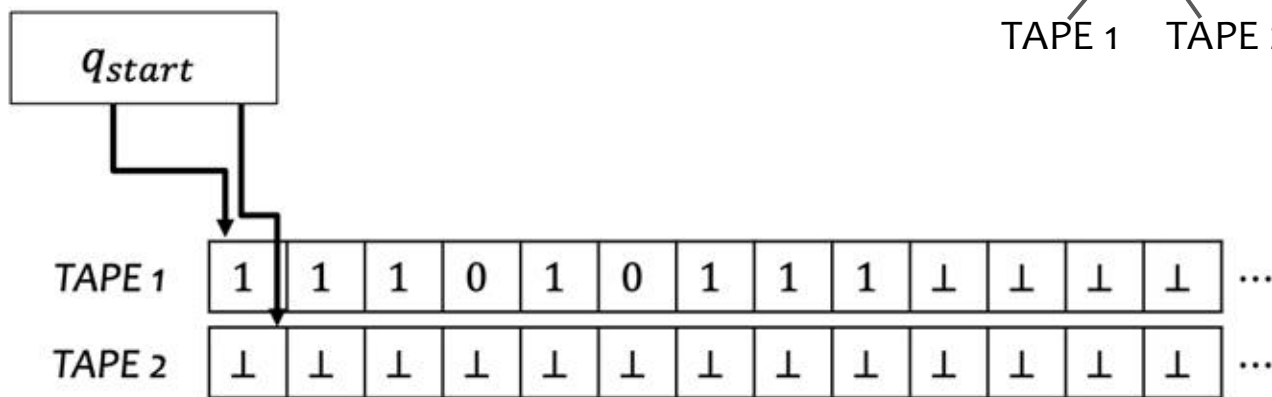
What if we added more tapes to a TM?

Well, the Church-Turing Thesis says there's no “computer” more powerful than a 1-tape TM.

So a 2-tape TM better not be more powerful.
Let's prove it!

Example transition rule for 2-tape TM: $\delta(q_1, 1, \perp) = (q_2, 0, 1, R, \dot{L})$

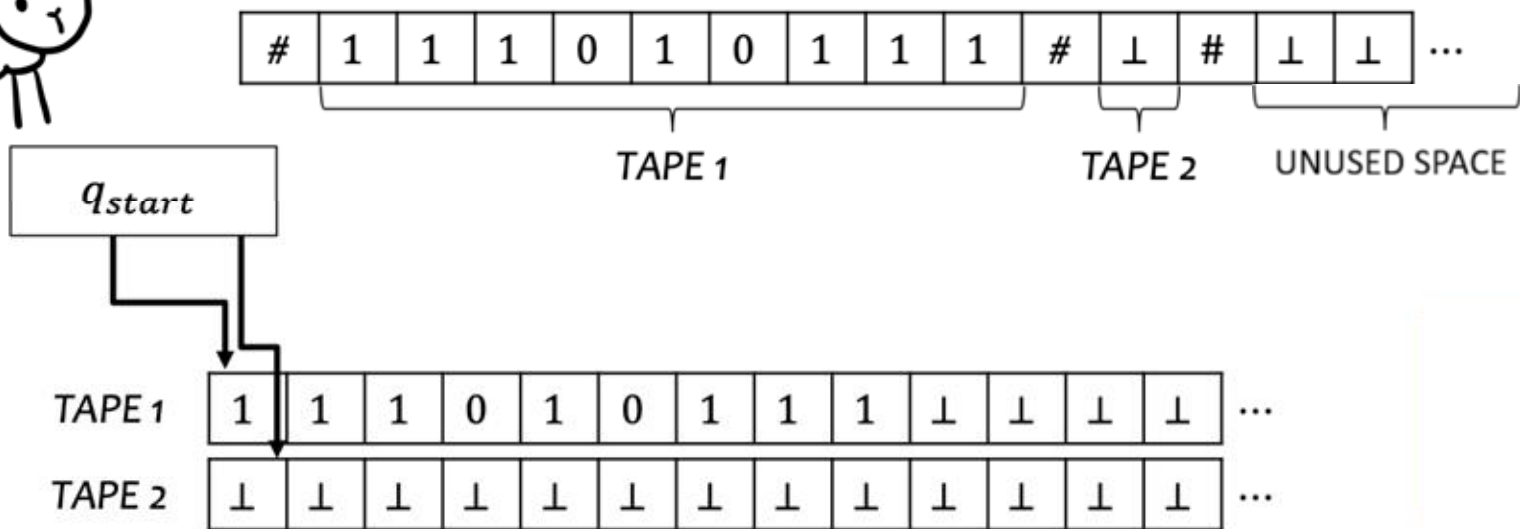
Diagram illustrating the transition rule: The input string "111010111" is on TAPE 1, and the rest of the tape is blank ($\perp$). The head is positioned at the first '1' on TAPE 1. The rule specifies that the machine transitions to state q_2 , writes '0' on TAPE 1, writes '1' on TAPE 2, moves the head on TAPE 1 to the right (R), and moves the head on TAPE 2 to the left (L).



Want to show: For any given 2-tape TM, there is a 1-tape TM that “simulates” its execution. **How?**

Similar reasoning shows that a TM with a 1-way infinite tape is equivalent to a TM with a 2-way infinite tape (used in turingmachine.io)

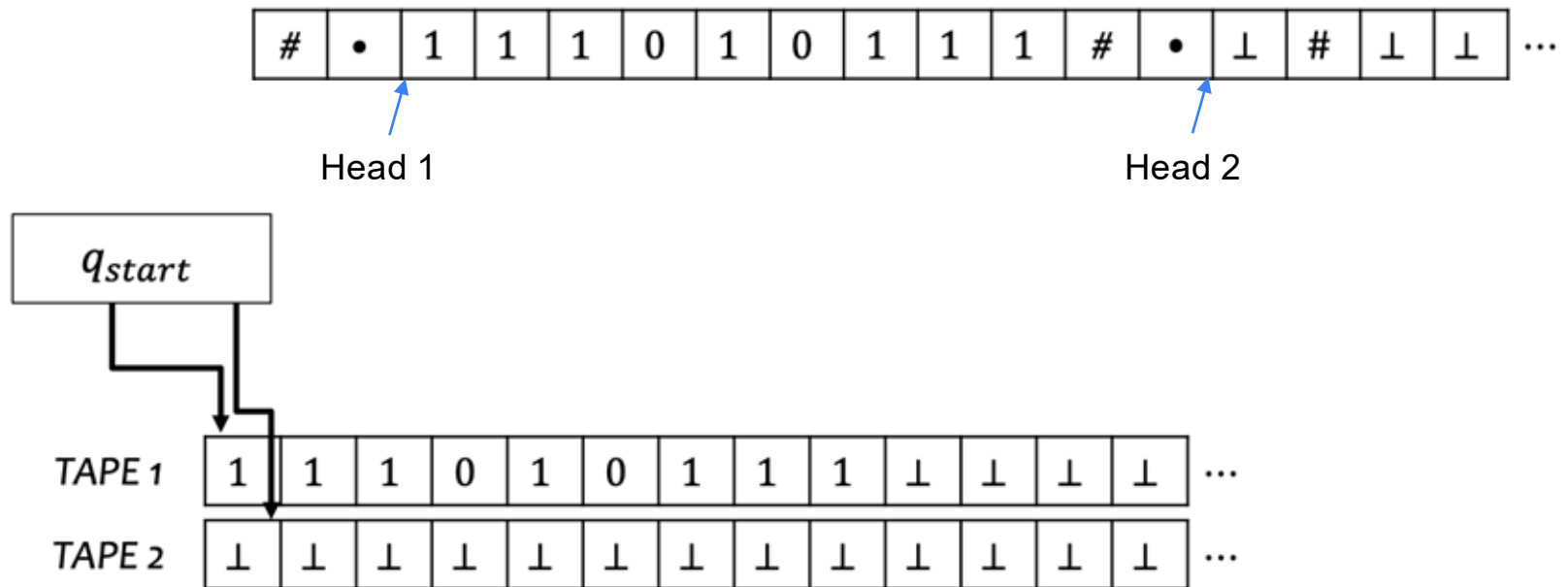
Issue 1: How do we store 2 tapes on only 1 tape?



Equivalence of 1-tape and 2-tape TMs

Issue 2: How do we keep track of 2 heads when we only have 1?

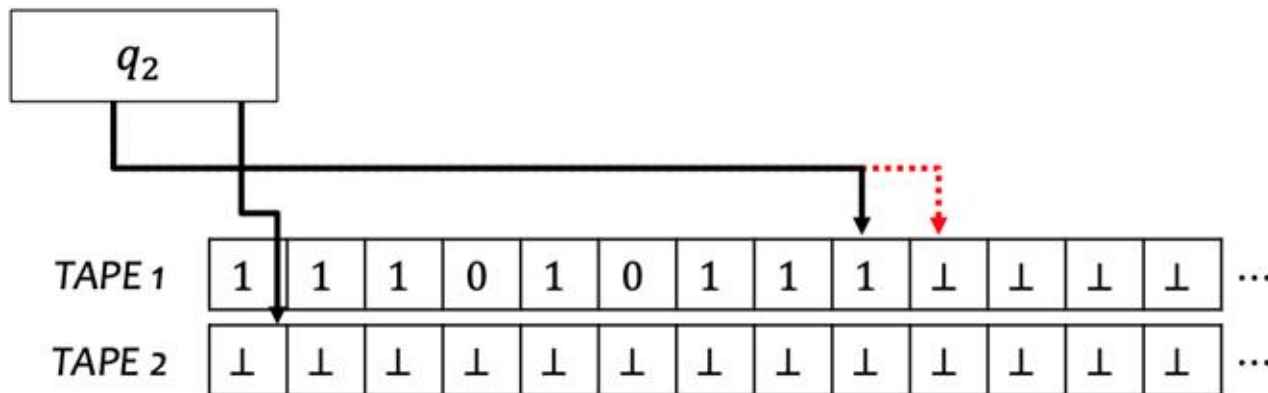
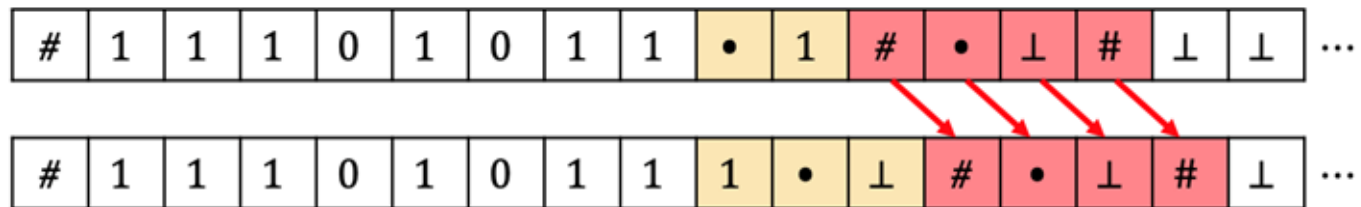
Idea: Need to demarcate location of each head – use a • symbol to the left of the cell where the head is.



Equivalence of 1-tape and 2-tape TMs

Issue 3: What if one of the heads moves past the recorded end of its tape?

Idea: Move the (finite) marked cells after it to the right 1 cell



Equivalence of 1-tape and 2-tape TMs

Issue 4: To know which transition rule to use, I need to know the current symbols of **both** tapes of the 2-tape TM, but I only have 1 head on my 1-tape TM.

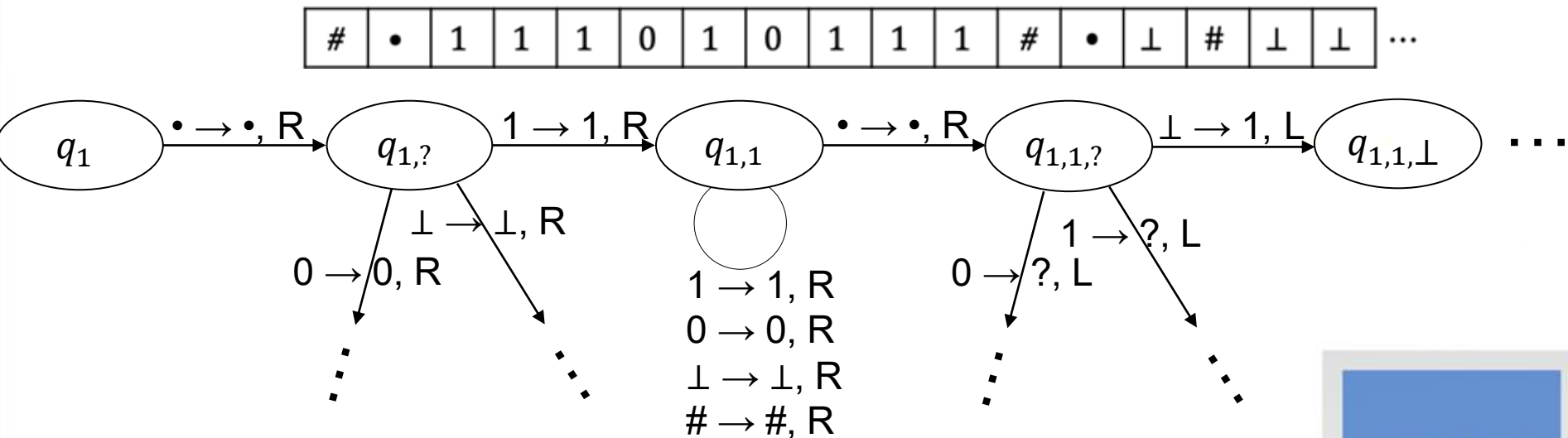
#	•	1	1	1	0	1	0	1	1	1	#	•	⊥	#	⊥	⊥	...
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	-----

$$\delta(q_1, 1, \perp) = (q_2, 0, 1, R, L):$$

#	•	1	1	1	0	1	0	1	1	1	#	1	•	⊥	#	⊥	...
#	0	•	1	1	0	1	0	1	1	1	#	•	1	1	#	⊥	...

Equivalence of 1-tape and 2-tape TMs

Issue 4: To know which transition rule to use, I need to know the current symbols of **both** tapes of the 2-tape TM, but I only have 1 head on my 1-tape TM.



$$\delta(q_1, 1, \perp) = (q_2, 0, 1, R, L):$$

